<https://www.google.ca>

google

TRANSFORMER NEDİR?

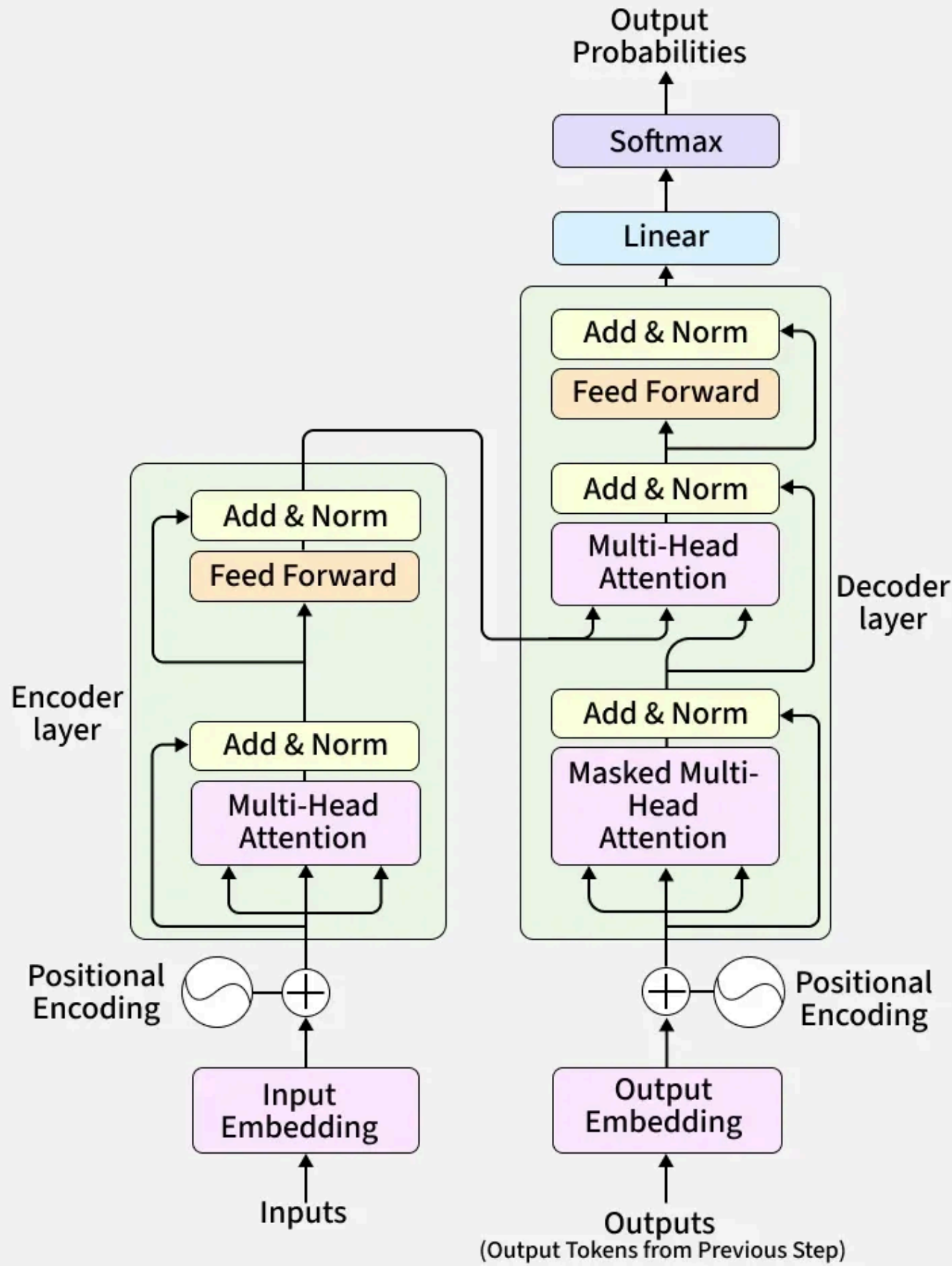


Nedir bu Transformer Modeli?

Transformer modeli, cümledeki kelimeleri sıralı verilerdeki ilişkileri izleyerek bağlamı ve dolayısıyla anlamı öğrenen bir sinir ağıdır. Transformer modelleri, bir dizideki birbirini etkileyen ve birbirine bağlı olan birbirinden uzak veri öğelerinin bile ince bağlantılarını tespit etmek için, attention veya self-attention adı verilen ve hala gelişmekte olan bir dizi matematiksel teknikleri uygulamaktadır.

İlk olarak Google'ın 2017 tarihli bir makalesinde tanımlanan transformer modelleri, bugüne kadar icat edilmiş en yeni ve en güçlü model yapılarından biridir. Bu model yapıları makine öğrenmesi alanındaki gelişmelerin öncüsü haline geldiği için bazı kesimler tarafından "dönüştürücü AI" olarak da adlandırılmaktadır.

Transformer Architecture



1. Girdi ve Gömme (Inputs & Input Embedding) Sürecin en başında, özetlenecek hikâye metnimizdeki kelimeler modele ham haliyle girer. Ancak yapay zekâ sözel metinleri okuyamadığı için bu kelimeler öncelikle devasa bir sözlükten karşılıkları bulunarak katmanına sokulur. Burada her bir kelime, sadece kendi anlamsal özelliklerini taşıyan, vektöre dönüştürülür.

2. Konumsal Kodlama (Positional Encoding) Anlamını bulan bu kelime vektörleri, Transformatör mimarisine hepsi aynı anda gireceği için model kelimelerin cümle içindeki sırasını doğal olarak bilemez. Her kelimenin anlamsal vektörünün üzerine, o kelimenin cümledeki sırasını ifade eden ve sinüs/kosinüs dalgalarıyla hesaplanmış eşsiz bir "konum barkodu" matematiksel olarak toplanır. Bu sayede kelimelerin hem ne anlama geldiği hem de hikâyedeki yeri sisteme tanıtılmış olur.

3. Kodlayıcı Bloğu (Encoder Layer) Bu bloğun kalbi olan "Multi-Head Attention" katmanında kelimeler birbirleriyle etkileşime girer; örneğin bir zamirin hikâyedeki hangi karakteri işaret ettiği burada çözülür. Ardından bu karmaşık ilişkiler "Feed Forward" katmanında filtrelenip sindirilir. Her katman geçişinde yer alan "Add & Norm" işlemleri ise sayıların aşırı büyüüp sistemi bozmasını engelleyerek veriyi dengede tutar. Kodlayıcı bloğu, tüm hikâyeyi baştan sona okuyup onun derin bağlamsal matematiğini çıkaran uzman bir okuyucu gibidir.

Kod Çözücü Bloğu (Decoder Layer) Kodlayıcı'nın çıkardığı bu matematiksel özeti alıp bizim anlayacağımız yeni ve kısa bir metne dönüştüren "yazar" kısmıdır. Üretim sürecine başlarken "Masked Multi-Head Attention" katmanı sayesinde sadece kendi ürettiği geçmiş kelimelere bakar. Hemen bir üstündeki ikinci "Multi-Head Attention" katmanında ise okların sol taraftan (Encoder'dan) geldiğini görürüz; işte burası, Kod Çözücü'nün yepyeni bir özet kelimesi üretirken orijinal hikâyenin bağlamından kopmamak için Kodlayıcı'nın çıkardığı ana fikre bakıp "kopya çektiği" yerdir.

Çıktı Katmanı (Linear & Softmax) Kod Çözücü bloğundan çıkan son hesaplamalar, şemanın en tepesindeki "Linear" ve "Softmax" katmanlarına ulaşır. Linear katman bu veriyi modelin bildiği on binlerce kelimelik sözlük boyutuna genişletir. Softmax katmanı ise bu devasa kelime havuzundaki her bir kelimeye 0 ile 1 arasında bir olasılık puanı atar. O anki bağlama göre en yüksek olasılığa sahip olan kelime (Output Probabilities) seçilir ve özetin sıradaki kelimesi olarak ekrana yazdırılır. Bu süreç, anlamlı ve tutarlı bir özet metni tamamlanana kadar döngü halinde devam eder.

EMBEDDING+ATTENTION

Word Embeddings

Sözel ifadelerin matematiksel olarak ifade edilmesi için kullanılan tekniklerdir. One Hot Encoding, TF-IDF, Word2Vec, FastText, yöntemleri sıklıkla kullanılan Word Embedding yöntemleridir. Verilerin durumu, boyutu ve işlenme amacına göre bu tekniklerden birisi (bazı durumlarda birkaçı) tercih edilerek kullanılır.



Embedding boyutu (d_{model}), bir doğal dil işleme modelinde her bir token'ın (kelime ya da alt-kelime parçasının) kaç boyutlu bir vektörle temsil edildiğini ifade eder. Modeller kelimeleri doğrudan anlayamaz; bunun yerine her kelimeyi sayılardan oluşan bir vektöre dönüştürür. İşte bu vektörün uzunluğu embedding boyutudur. Örneğin $d_{\text{model}} = 512$ ise, her token 512 tane sayıdan oluşan bir vektörle temsil edilir. Bu sayılar kelimenin anlamına dair farklı özellikleri (bağlamsal kullanım, benzerlik ilişkileri, soyutluk derecesi vb.) kodlar. Transformer mimarisinde embedding katmanı, attention katmanı ve diğer temel katmanlar aynı boyutta çalıştığı için bu ortak temsil boyutuna d_{model} adı verilir. d_{model} büyüdükçe model daha fazla anlamsal bilgiyi temsil edebilir ancak parametre sayısı ve hesaplama maliyeti de artar. Küçük d_{model} daha hızlı ve hafif bir model sağlarken, büyük d_{model} daha güçlü fakat daha maliyetli bir yapı oluşturur.

TF-IDF

TF-IDF, kelimelerin dokümanlar içerisindeki matematiksel önemini belirlemeye yarayan istatistiksel bir ölçüdür. Vektörizasyon işlemi One Hot Encoding'e benzer şekilde gerçekleşir. Farklı olarak sözel ifadeye karşılık gelen değere 1 yerine TF-IDF değeri atanır. TF-IDF değeri TF ve IDF değerlerinin çarpımı sonucunda elde edilir.

ÖR:[Bu bir sayıdır],
[Bu bir kelimedir],
[Bu sayı ve kelime değildir]

Yukarıdaki örnekte "Bu" ifadesi 3 dokümanda da geçerken "bir" ifadesi 2 dokümanda ve "sayıdır" ifadesi de yalnızca bir dokümanda geçmektedir. Bunlara göre önce TF sonra IDF değerlerini sırasıyla bulalım.

TF (Term Frequency)

En basit ifadeyle terim sıklığı, doküman içindeki hedef terim sayısının dokümandaki toplam terim sayısına oranıdır. Yukarıdaki örneğe göre TF değerleri hesaplanacak olursa;

[0.33, 0.33, 0.33],
[0.33, 0.33, 0.33],
[0.20, 0.20, 0.20, 0.20, 0.20] şeklinde olacaktır.

IDF (Inverse Document Frequency)

IDF değeri, toplam doküman sayısının hedef terimin geçtiği doküman sayısına olan oranının logaritmasıdır. Bu aşamada doküman içerisinde terimin kaç defa geçtiğinin bir önemi yoktur. Geçip geçmediğinin tespit edilmesi yeterlidir. Bu örnekte alınacak logaritmanın taban değeri 10 olarak belirlenmiştir. Ancak farklı değerlerin kullanılmasında da herhangi bir sakınca yoktur.

"Bu": $\log(3/3) = 0$,
"bir": $\log(3/2) = 0.1761$,
"kelimedir, sayıdır vb.": $\log(3/1) = 0.4771$

Böylelikle hem TF hem de IDF değerleri elde edilmiştir. Bu değerler ile vektörizasyon işlemi yapılacak olursa öncelikle her bir doküman için bütün dokümanlardaki unique kelime sayısı kadar elemandan oluşan bir vektör oluşturulur (Bu örnekte 8 terim vardır). Bu aşamada çözülmesi gereken bir problem vardır. "Bu" teriminde de görüldüğü üzere IDF değerinin 0 olmasından dolayı TF-IDF değeri de sıfır olacaktır. Ancak vektörizasyon işlemi sırasında doküman içerisinde geçmeyen kelimelere de (örnek olarak 1.cümlede "kelimedir" ifadesi geçmemektedir) 0 değeri atanacaktır. Buradaki karışıklığın önüne geçebilmek adına vektörleştirme sırasında TF-IDF değerlerine smoothing işlemi uygulanır. En yaygın kullanılan yöntem elde edilen değerlere 1 eklenmesidir. Amaca göre daha sonra bu değerlere normalizasyon işlemi de uygulanabilir. Yukarıda anlatılanlara göre vektörizasyon işlemi yapılacak olursa;

TF - IDF

$$TF-IDF = TF(t,d) \times IDF(t)$$

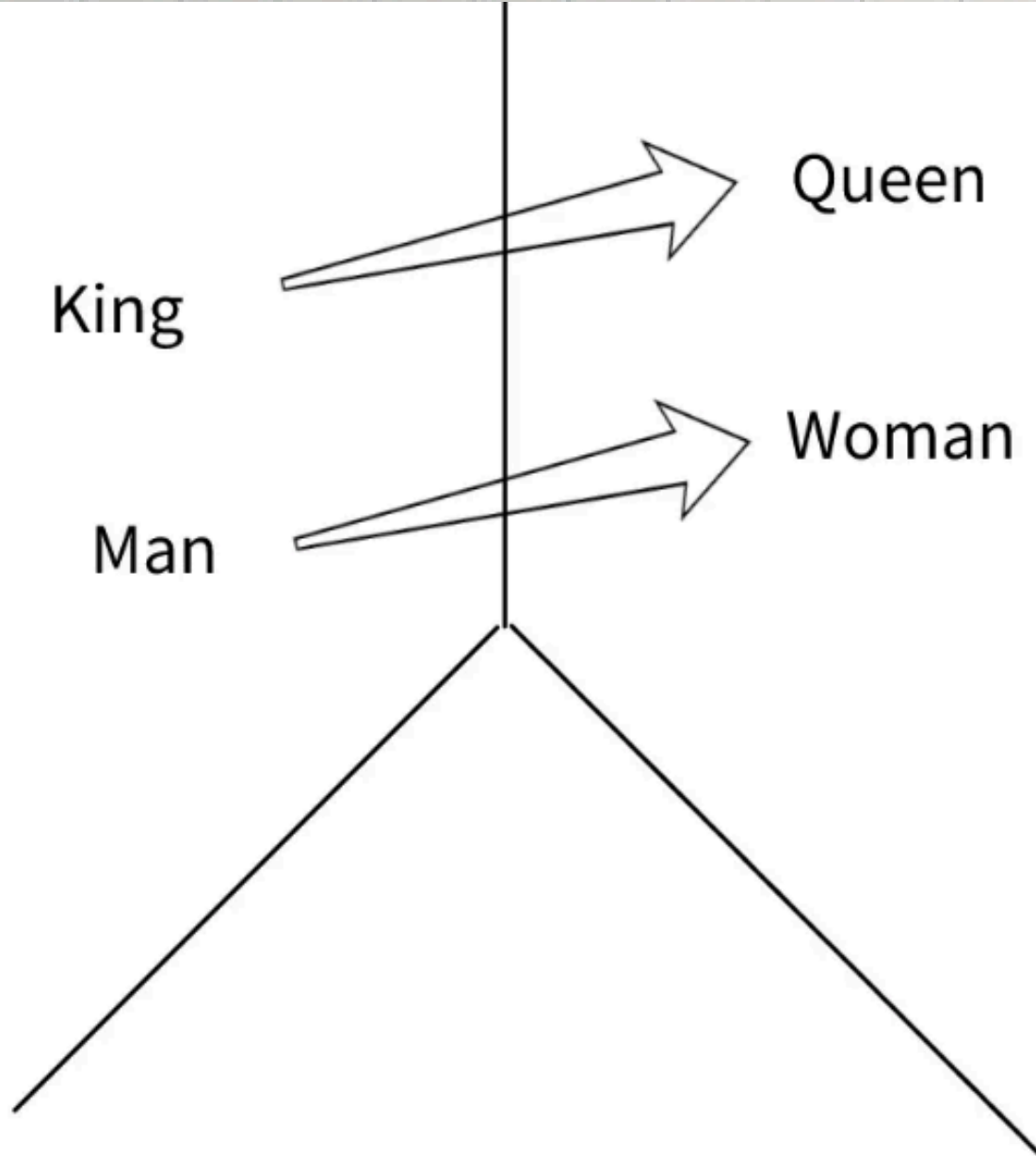
↑ ↑
Term frequency Inverse document frequency

1.NESİL:Bu eski yöntemler kelimelerin "anlamını" veya "bağlamını" yakalayamaz. Transformer'lar, sadece istatistik saymak yerine kelimelerin anlamını derinlemesine öğrenmek üzere tasarlanmıştır. Bu yöntemler bu işi sağlamaz.

[1. , 1.1761 , 1.4771 , 0. , 0. , 0. , 0. , 0.],
[1. , 1.1761 , 0. , 1.4771 , 0. , 0. , 0. , 0.],
[1. , 0. , 0. , 0. , 1.4771 , 1.4771 , 1.4771 , 1.4771], sonuçları elde edilir.

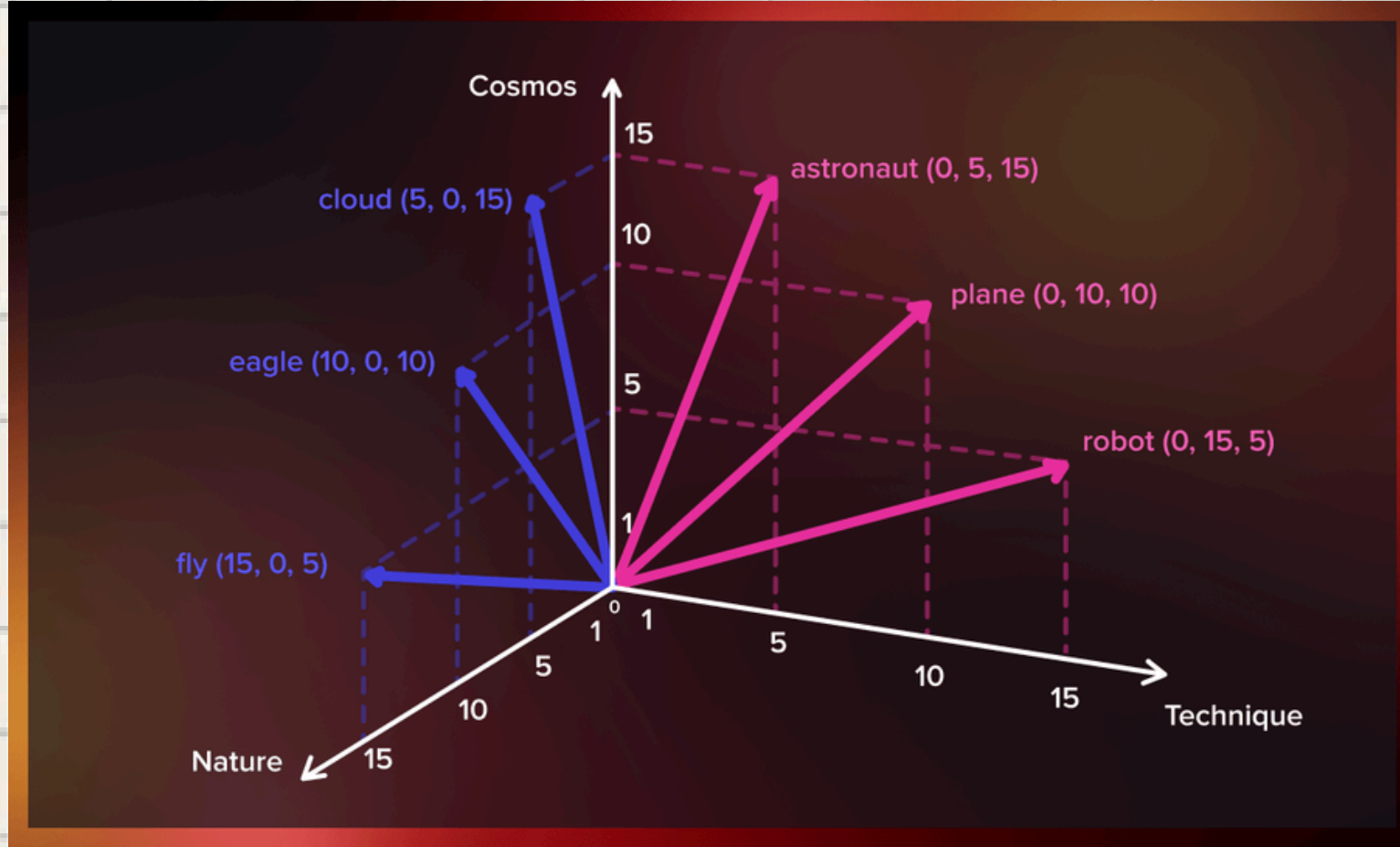
Word2Vec

Word2vec, sıklıkla kullanılan word embedding tekniklerinden bir diğeridir. Bütün corpus taranarak hedef kelimenin daha çok hangi kelimelerle birlikte geçtiği tespit edilerek vektör oluşturma işlemi gerçekleştirilir[3]. Bu sayede kelimelerin anlamsal olarak birbirlerine yakınlıkları da ortaya çıkmış olur. Örnek vermek gerekirse her harf bir kelimeyi temsil edecek şekilde $..x\ y\ A\ z\ w..$ ve $..x\ y\ B\ z\ k..$ dizilimindeki A ve B kelimelerinin birbirlerine olan yakınlıkları, bu kelimelerin $..x\ l\ C\ d\ m..$ dizilimindeki C kelimesine olan yakınlıklarından daha fazladır. Vektör oluşumunda bu durum göz önünde bulundurulduğu için kelimeler arasındaki anlamsal yakınlık matematiksel olarak ifade edilmiş olur.

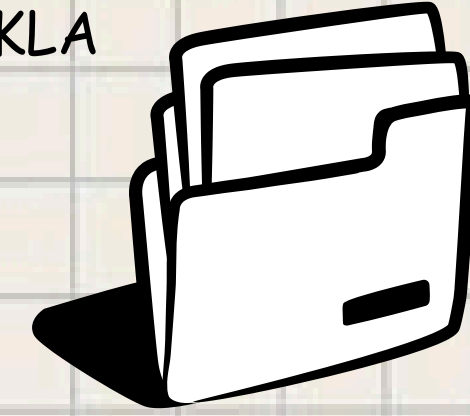
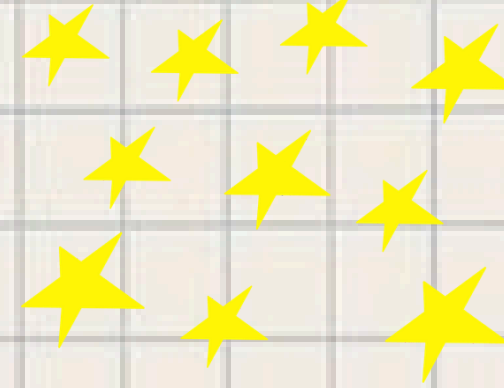


Word2Vec konularında en sık kullanılan görsellerden birisi görülmektedir. Bu kelimeler arasındaki anlamsal yakınlık vektörel değerlerin birbirlerine olan yakınlığından başka bir şey değildir. Hatta sıklıkla verilen örneklerden birisi de "king- man + woman = queen" denklemidir. Burada gerçekleşen olay vektörlerin birbirlerinden çıkartılması ve toplanması sonucunda elde edilen vektörel değer "queen" ifadesine karşılık gelen vektöre eşit olmasıdır. Buradan "king" ve "queen" kelimelerinin birbirlerine çok benzer olduğu ancak yalnızca cinsiyetleri sebebiyle vektörel farklılıkların ortaya çıktığı anlaşılabılır.

Word2Vec yönteminde One Hot Encoding ve TF-IDF yöntemlerinden farklı olarak unsupervised learning işlemi gerçekleştirilir. Etiketsiz veriler yapay sinir ağları aracılığıyla eğitilerek kelime vektörleri üreten Word2Vec modeli oluşturulur. Yine diğer yöntemlerden farklı olarak vektör boyutu corpus içerisindeki unique kelimelerin sayısı kadar olmaz. Burada vektörün boyutu corpus büyüklüğüne ve proje türüne göre seçilebilir. Bu da özellikle çok büyük verilerde büyük fayda sağlamaktadır. Örnek olarak büyük bir corpus içerisinde 300 000 adet unique kelimenin olduğunu varsayarsak One Hot Encoding ile vektör oluşturma işlemi yapıldığında her bir kelime için 300 000 boyutlu yalnızca bir elemanın değeri 1 diğerlerinin 0 olan bir vektör oluşturulması gerekir. Ancak Word2Vec işleminde vektör boyutu 300 (kullanıcı seçimine bağlı olarak daha fazla veya daha az da olabilir) seçilerek gereksiz büyük boyutlu vektörel işlemlerden kaçınılmış olur.



MODELİ İNCELEMELİK İÇİN TIKLA



2.NESİL: Word2Vec statiktir. Yani "Yüz" kelimesi için her zaman aynı vektörü üretir. "Suda yüz" cümlesi ile "Yüzüm ağrıyor" cümlesindeki "yüz" kelimesini ayırt edemez.

Input Embeddings

Derin öğrenme modellerinde, input embeddings , kelimeler veya tokens gibi ham verileri model tarafından işlenebilecek sayısal gösterimlere dönüştürmek için kullanılır. Transformer algoritmaları bağlamında, input embeddings çok önemlidir çünkü bir dizideki her kelimeyi veya tokenı yüksek boyutlu bir vektöre dönüştürürler. Bu vektör, kelimenin anlamını ve semantik özelliklerini modelin anlayabileceği ve işleyebileceği bir şekilde temsil eder.

Nasıl Çalışıyor?

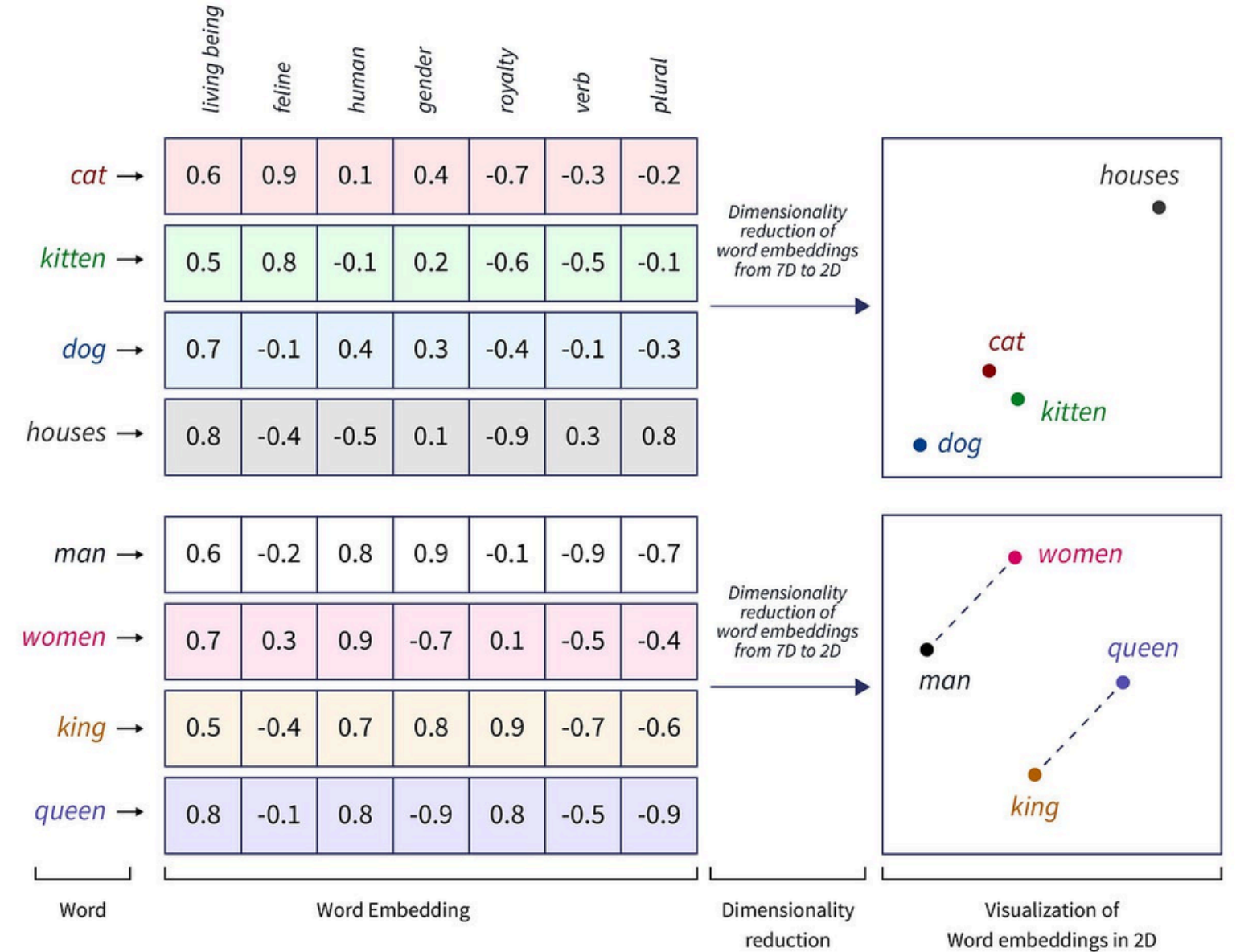
Transformer modelinde, input embedding tipik olarak Word2Vec , GloVe gibi önceden eğitilmiş embedding kullanılarak veya eğitim sırasında sıfırdan öğrenilen embedding kullanılarak oluşturulur. Dizideki her kelime, sabit boyutta bir vektöre eşlenir. Bu vektörler, benzer kelimelerin vektör uzayında birbirine daha yakın gömülerine sahip olacak şekilde öğrenilir.

İşte girdi input embedding nasıl çalıştığına dair bir açıklama:

Tokenizasyon : İlk olarak, giriş metni daha küçük parçalara (kelimeler, alt kelimeler veya karakterler) ayrılır.

Lookup Table: : Bir lookup table, aynı zamanda embedding matrix olarak da adlandırılır ve sözlükteki her bir belirteç için embedding vector içerir. Her tokena, genellikle yüksek boyutlu (örneğin, 512 veya 1024) bir embedding vector atanır.

Embedding Layer: : Tokenleştirilmiş dizi, her bir tokeni arama tablosundan karşılık gelen embedding vector eşleyen bir embedding layer geçirilir.



Mathematics Behind It:

$$\text{Embedding}(t_i) = E[t_i]$$

Where:

- E is the **embedding matrix** of size $V \times d_{\text{model}}$, where V is the vocabulary size and d_{model} is the embedding dimension.
- t_i is the token index in the vocabulary for the i -th token in the input sequence.
- $E[t_i]$ represents the **embedding vector** corresponding to the token t_i , which is a row of the embedding matrix E .

t_i (Token Index): Cümledeki i . kelimenin, sözlükteki sıra numarası ID'si demektir. Örneğin modelin sözlüğünde "Elma" kelimesi 405. sıradaysa, $t_i = 405$ olur.

E:(Embedding Matrix): Bu, modelimizin sahip olduğu o devasa "Kelimeler ve Anlamları" tablosudur. Boyutu $V \times d$ kadardır.

V:(Vocabulary Size): Modelin bildiği toplam kelime (veya hece/token) sayısıdır (Örn: 50.000 satır).

d_{model} : Her bir kelimeyi ifade etmek için kullandığımız standart vektör uzunluğudur (Örn: 512 sütun).

Input Embedding İşleminin Temel Özellikleri:

Sabit Gösterim : Her bir belirteç, eğitim ve çıkarım boyunca sabit kalan, sabit boyutlu bir vektöre eşlenir.

Eğitim Sırasında Öğrenilen : Gömülü vektörler rastgele başlatılır ve daha sonra modelin eğitildiği belirli görevin inceliklerini yakalamak için eğitim sırasında ince ayar yapılır.

Anlamsal Benzerlik : Benzer anlamlara sahip kelimelerin benzer gömülü temsilleri olma eğiliminde olması, modelin kelimeler arasındaki ilişkileri yakalamasını kolaylaştırır.

```
import torch
import torch.nn as nn

class InputEmbedding(nn.Module):
    def __init__(self, vocab_size, embed_dim):
        """
        Initializes the InputEmbedding layer.

        Parameters:
        vocab_size (int): Size of the vocabulary (i.e., the number of unique tokens).
        embed_dim (int): Dimensionality of the embedding space (i.e., size of the embedding vectors).
        """
        super(InputEmbedding, self).__init__()

        # Embedding layer for converting tokens to dense vectors
        self.embedding = nn.Embedding(vocab_size, embed_dim)

    def forward(self, x):
        """
        Forward pass to get the embedding representation of input tokens.

        Parameters:
        x (Tensor): A tensor of token indices with shape (batch_size, seq_len), where
        each value represents a token's index in the vocabulary.

        Returns:
        Tensor: A tensor of shape (batch_size, seq_len, embed_dim) representing
        the input tokens' embeddings.
        """
        return self.embedding(x)
```

Örnek Kullanım :

```
# Define parameters
vocab_size = 10000 # Example vocabulary size
embed_dim = 512    # Embedding dimension

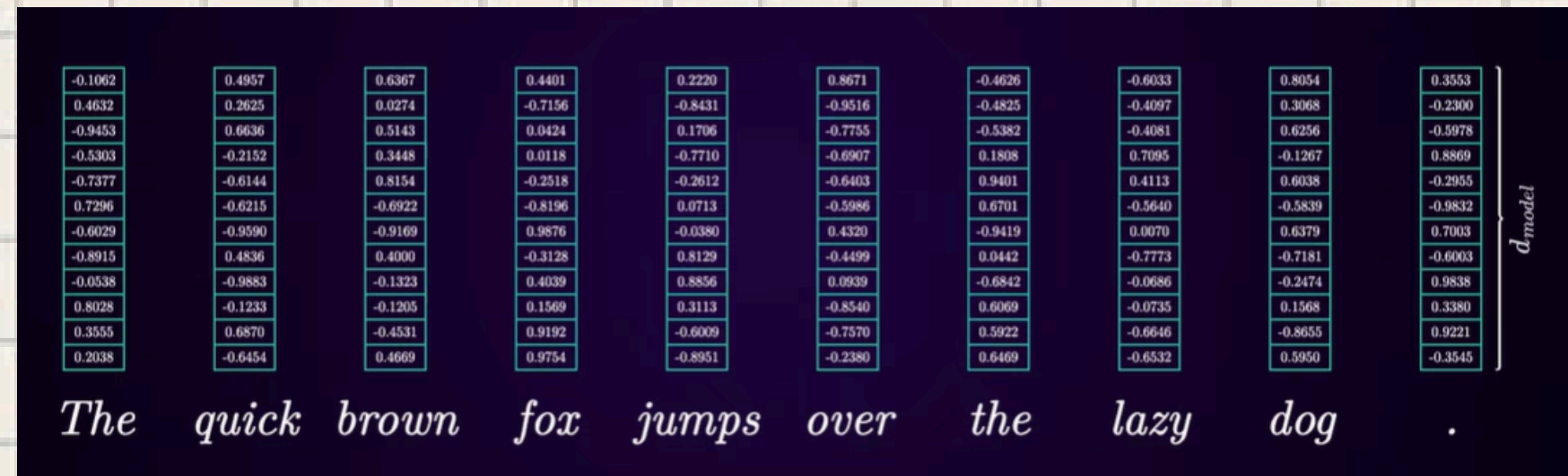
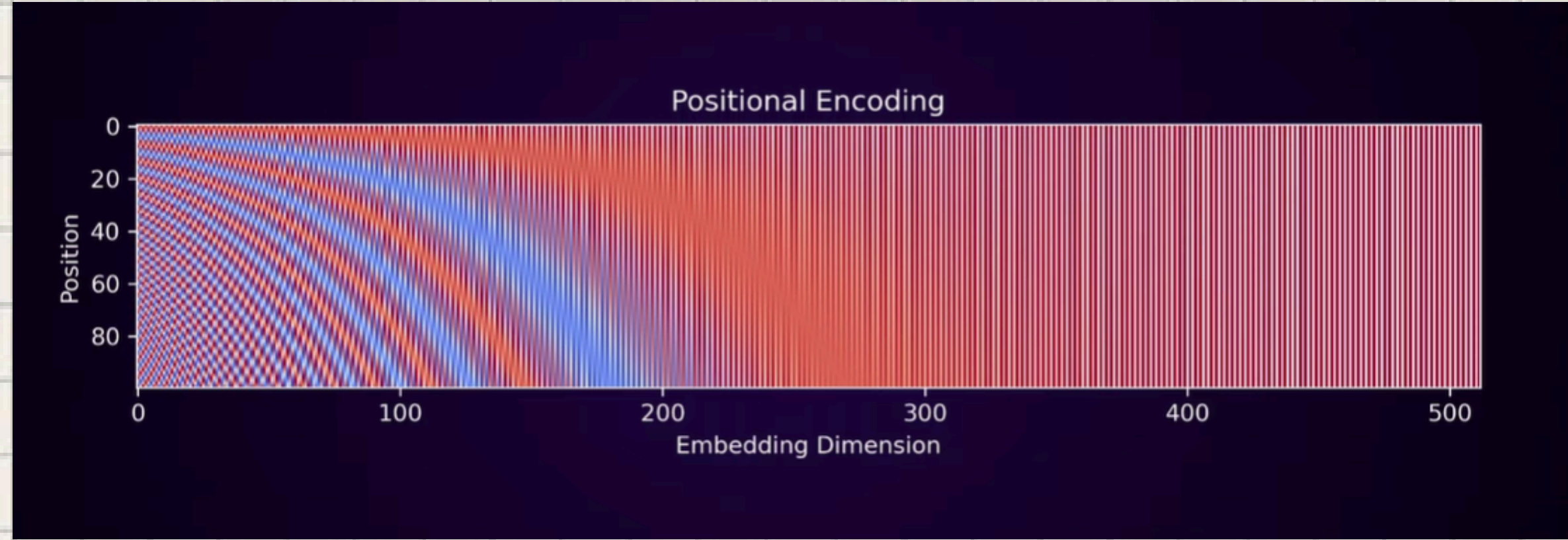
# Initialize the InputEmbedding class
embedding_layer = InputEmbedding(vocab_size, embed_dim)

# Example input tensor (batch of token indices)
# Shape: (batch_size, seq_len), where each element is a token index
example_input = torch.tensor([[1, 5, 8], [2, 6, 9]]) # Batch of 2 sequences with 3 tokens each

# Get the embeddings
output_embeddings = embedding_layer(example_input)

# Print the shape of the output
print("Output Embeddings Shape:", output_embeddings.shape)
```


POSITIONAL ENCODING



Eski dil modelleri (RNN'ler gibi) kelimeleri cümlede sırayla, tek tek okurdu. Bu yüzden kelimenin sırasını doğal olarak bilirlerdi. Ancak Transformer'lar hızı artırmak için bütün kelimeleri aynı anda (paralel olarak) okur.

Bütün kelimeler aynı anda modele girince, modelin "Köpek adamı ısırıldı" ile "Adam köpeği ısırıldı" cümleleri arasındaki farkı anlaması imkansız hale gelir; çünkü iki cümlede de aynı kelimeler var. Modele kelimelerin sadece anlamlarını değil, konumlarını (hangi sırada olduklarını) da söylememiz gerekir.

pos | 0 1 2 $d_{model} = 3$

I love cats...

I $\begin{bmatrix} 0 & 1 & 2 \\ 0.1 & 0.2 & 0.1 \end{bmatrix}$

love $[0.3, 0.22, 1.2]$

Cats $[0.11, 0.20, 0.4]$

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10,000^{\frac{2i}{d_{model}}}}\right)$$

$$PE_{(p, 2i+1)} = \cos\left(\frac{pos}{10,000^{\frac{2i}{d_{model}}}}\right)$$

pos | 0 1 2 $d_{model} = 3$ $\cos\left(\frac{1}{10000}\right)$

I love cats...

I $\begin{bmatrix} 0 & 1 & 2 \\ 0.1 & 0.2 & 0.1 \end{bmatrix}$

love $[0.3, 0.22, 1.2]$

Cats $[0.11, 0.20, 0.4]$

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10,000^{\frac{2i}{d_{model}}}}\right)$$

$$PE_{(p, 2i+1)} = \cos\left(\frac{pos}{10,000^{\frac{2i}{d_{model}}}}\right)$$

$$PE_{(0,0)} = \sin\left(\frac{0}{10000^0}\right)$$

$$\sin 0 = 0$$

$$PE_{(0,3)} = \cos\left(\frac{1}{10000^{\frac{3}{2}}}\right)$$

1 $[0,$

loop

Cats

pos | 0 1 2 $d_{model} = 3$ $\cos\left(\frac{1}{10000^{0.67}}\right) =$

I love cats...

I $\begin{bmatrix} 0 & 1 & 2 \\ 0.1 & 0.2 & 0.1 \end{bmatrix} E$ $(E + PE)$

love $[0.3, 0.22, 1.2]$

Cats $[0.11, 0.20, 0.4]$

$$PE_{(0,4)} = \frac{2}{10000^{\frac{4}{3}}}$$

1 $[0, 0.99995, 0.000796] PE = \frac{2}{2511.886}$

loop

Cats

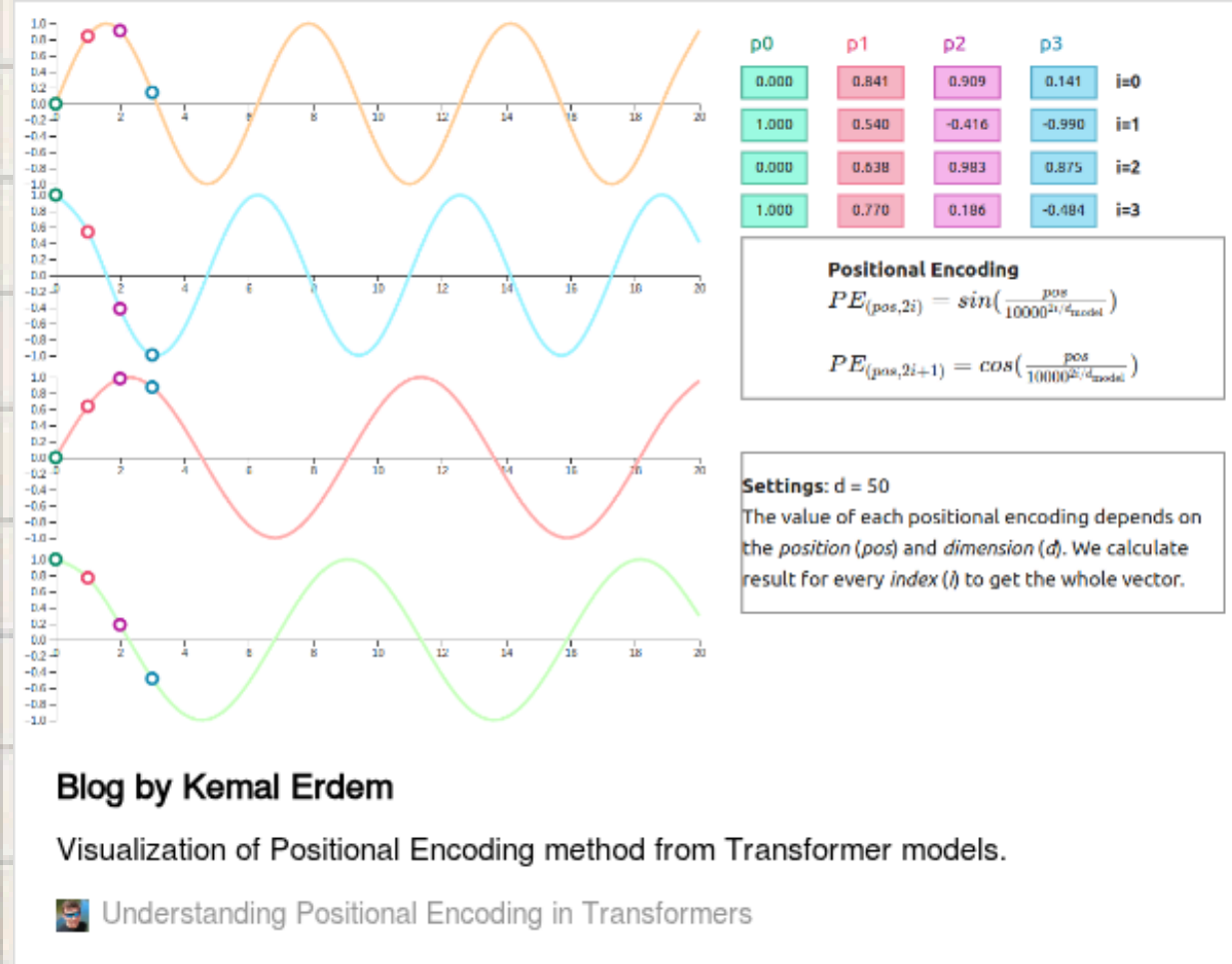
$$= 0.000796$$

Çift indexler için sin tek indexler için cos kullanılarak işlemler yapılır bulunan sonuçlarla E vektörü toplanır. Yanda örnek çözüm verilmiştir.

Araştırmacılar modelin bu konum sayılarını eğitim sırasında "kendi kendine öğrenmesi" (Learned Positional Embeddings) yerine, dışarıdan sabit bir matematiksel "formül" ile (Sinüs/Kosinüs) verilmesini iki devasa nedenden dolayı tercih etmiştir:

Görülmemiş Uzunluklara Uyum (Extrapolation): Eğer model konumları kendi öğrenseydi, eğitim sırasında sadece örneğin en fazla 512 kelimelik cümleler gördüyse, canlıya alındığında 600 kelimelik bir metin geldiğinde "Ben 600. sıranın ne demek olduğunu öğrenmedim" diyerek çökerdi. Ancak sabit bir sinüs/kosinüs formülü kullanıldığında, model eğitimde hiç görmediği kadar uzun metinlerle bile kusursuz çalışabilir çünkü formül sonsuza kadar hesaplanmaya devam edebilir.

Göreceli Konum (Relative Distance) Kolaylığı: Yapay zekanın aslında kelimenin "tam olarak 45. sırada" olmasından çok, "diğer kelimedenden 3 adım uzakta" olduğunu (göreceli konumu) bilmeye ihtiyacı vardır. Lisedeki trigonometri derslerinden hatırlayacağımız $\sin(a+b)$ gibi formül özellikleri sayesinde, Transformer mimarisi iki kelime arasındaki uzaklık mesafesini doğrusal dönüşümlerle çok kolay bir şekilde matematiksel olarak hesaplayabilir.

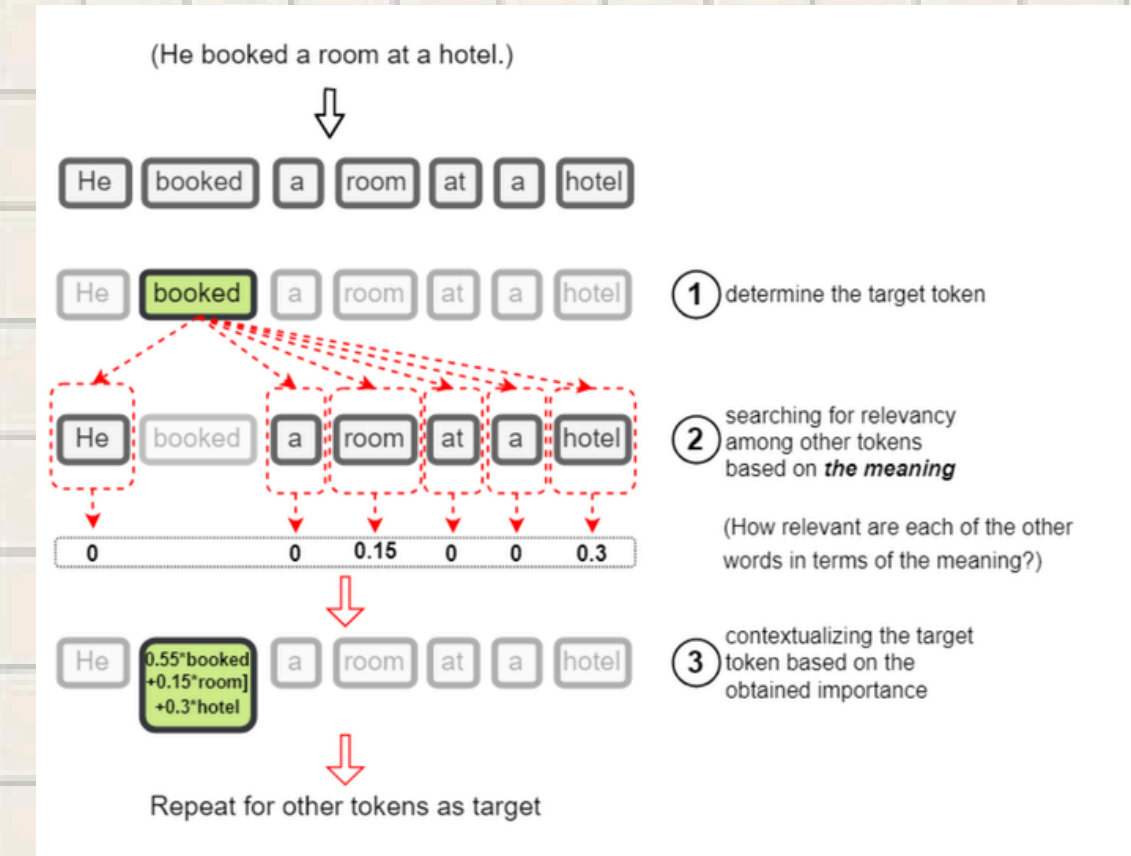


Attention

Makine öğreniminde attention, bir dizideki her bir bileşenin, aynı dizideki diğer bileşenlere göre ne kadar önemli olduğunu belirleyen bir yöntemdir. Doğal dil işleme bağlamında bu önem, bir cümledeki her kelimeye atanan "soft" ağırlıklar ile temsil edilir. Daha genel anlamda attention, sabit genişlikte bir dizi boyunca token embedding olarak adlandırılan vektörleri kodlar; bu dizinin uzunluğu tonlarca token'dan milyonlarca token'a kadar değişebilir.

"Hard" ağırlıkların aksine—ki bunlar geri yayılım (backward pass) sırasında hesaplanır—"soft" ağırlıklar yalnızca ileri geçişte (forward pass) mevcuttur ve bu nedenle girdinin her adımında değişir. Daha eski tasarımlarda attention mekanizması, seri çalışan bir tekrarlayan sinir ağı (RNN) tabanlı dil çeviri sistemine entegre edilmişti. Ancak daha yeni bir tasarım olan transformer, görece yavaş olan ardışık RNN yapısını ortadan kaldırmış ve bunun yerine çok daha hızlı olan paralel attention şemasına büyük ölçüde dayanmıştır.

İnsanlardaki dikkat (attention) süreçlerine dair fikirlerden ilham alan attention mekanizması, tekrarlayan sinir ağlarının gizli katmanlarındaki bilgiyi kullanmanın zayıf yönlerini gidermek amacıyla geliştirilmiştir. RNN'ler, bir cümledeki kelimelerde yer alan daha güncel bilgilere ağırlık verme eğilimindeyken, cümledeki bilgiler zamanla zayıflar. Attention ise bir token'ın, yalnızca önceki durumlar aracılığıyla değil, bir cümledeki herhangi bir bölümüne doğrudan ve eşit biçimde erişebilmesini sağlar.



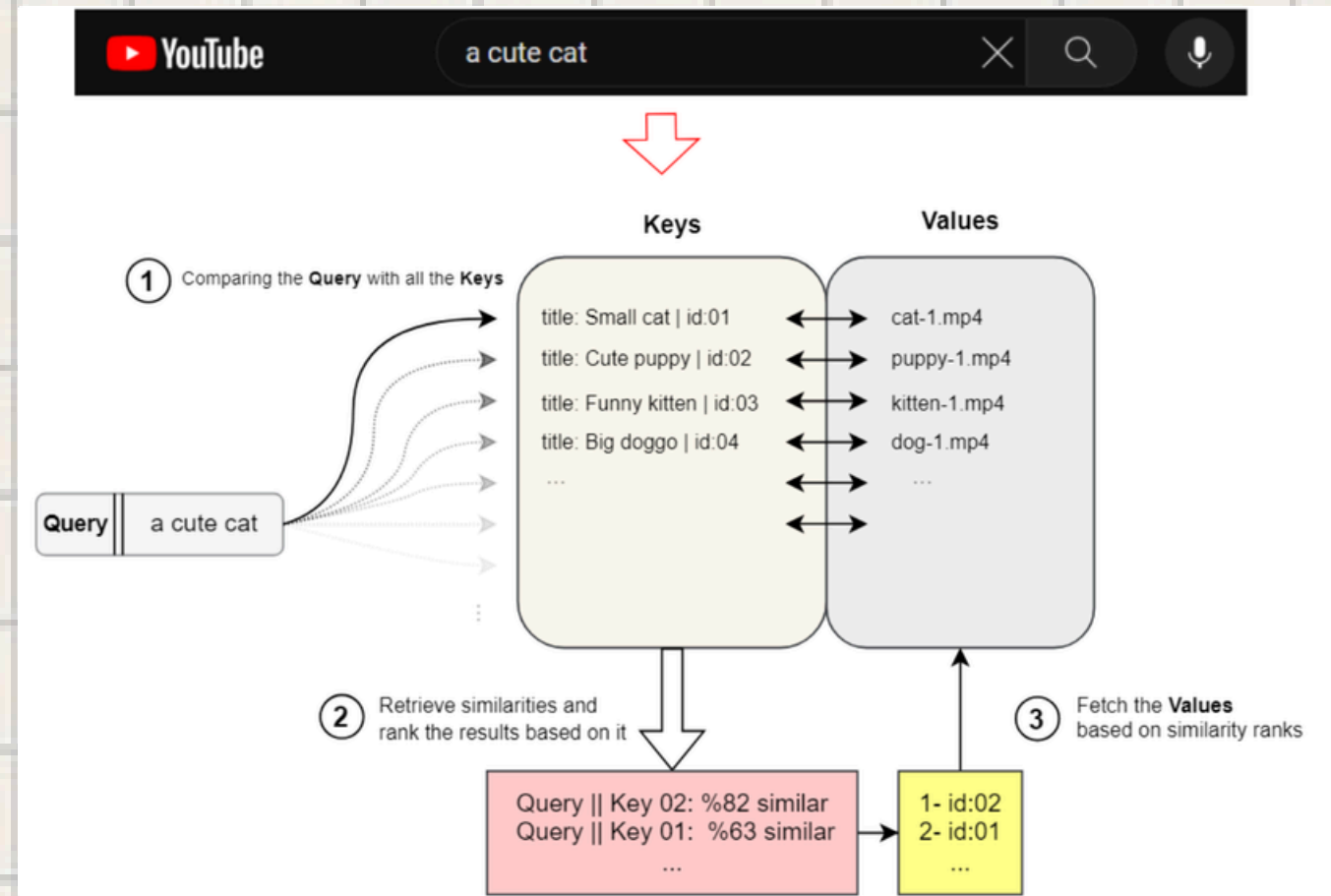
İşte Query, Key ve Value kavramlarının anlam kazandığı yer burasıdır.

Genel olarak attention mekanizmasının temelde şunları yaptığını söyleyebiliriz: hedef bir token'a olan ilgilerine veya önemlerine göre dizideki diğer token'lara ağırlık atamak ve onlardan ilgili bağlamı çıkarmak (yani onlara odaklanmak).

Ve bu ilgiyi/önemi bulabilmek için diziyi (sequence) taramamız ve token'ları birebir karşılaştırmamız gerekir.

Query, Key, and Value

Daha iyi anlaşılması için, örneğin YouTube'da bir arama yaptığınızı düşünün. YouTube'un tüm videolarını, "video başlığı" ve "video dosyasının" kendisinden oluşan bir çift olarak sakladığını varsayalım. Buna bir Key-Value çifti diyoruz; burada Key video başlığı, Value ise videonun kendisidir. Arama kutusuna yazdığınız metne ise arama terimleriyle Query denir. Dolayısıyla bir bakıma arama yaptığınızda YouTube, arama Query'nizi tüm videoların Key'leri ile karşılaştırır, ardından aralarındaki benzerliği ölçer ve Value'ları en yüksek benzerlikten aşağıya doğru sıralar.

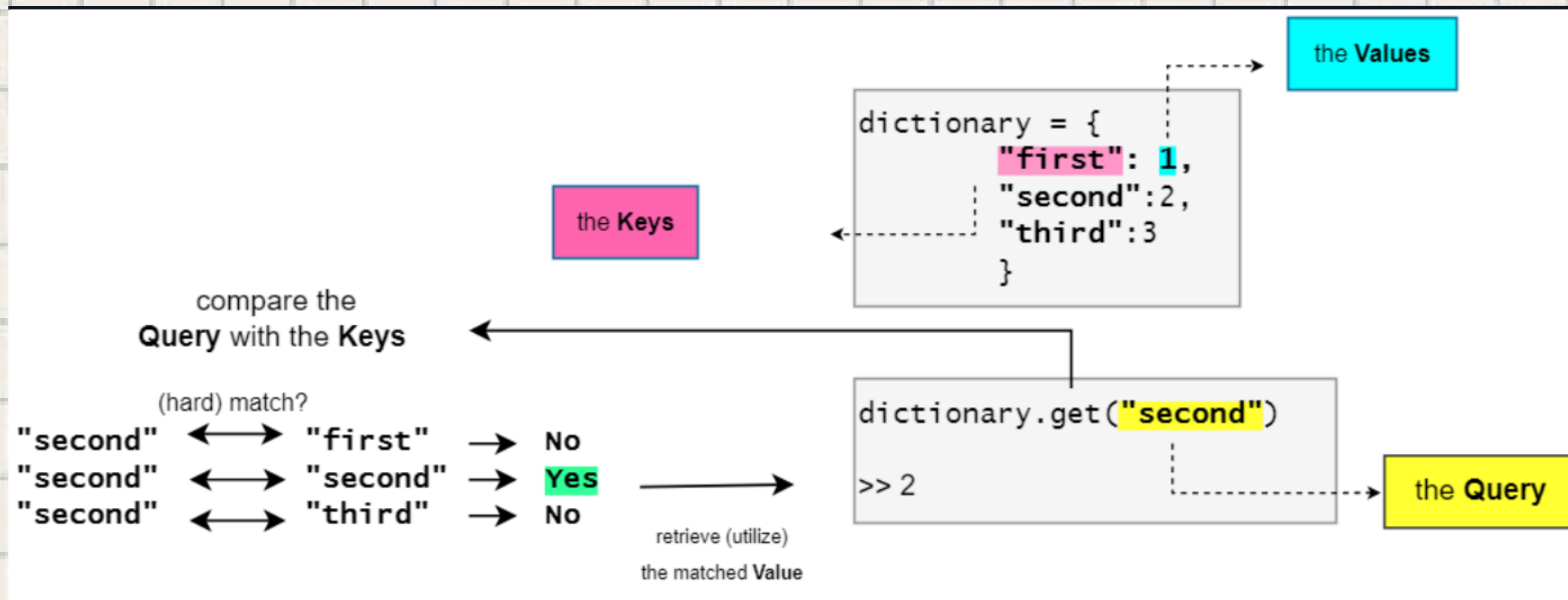


Bizim problemimizde, bir token vektörleri dizimiz (sequence) var ve dizideki her bir token (kelime) embedding'ini yeniden ağırlıklandırmak ve bağlama oturtmak için gereken ağırlıkları bulmak istiyoruz. Bunu şu şekilde düşünebiliriz:

- Aramak istediğiniz şey Query'dir.
- İçinde arama yaptığınız küme Key-Value çiftleridir.
- İlgi/önem/benzerlik derecesini ölçmek için Query, tüm Key'ler ile karşılaştırılır.
- Value'lar, atanan bu benzerlik ölçüsüne göre kullanılır.

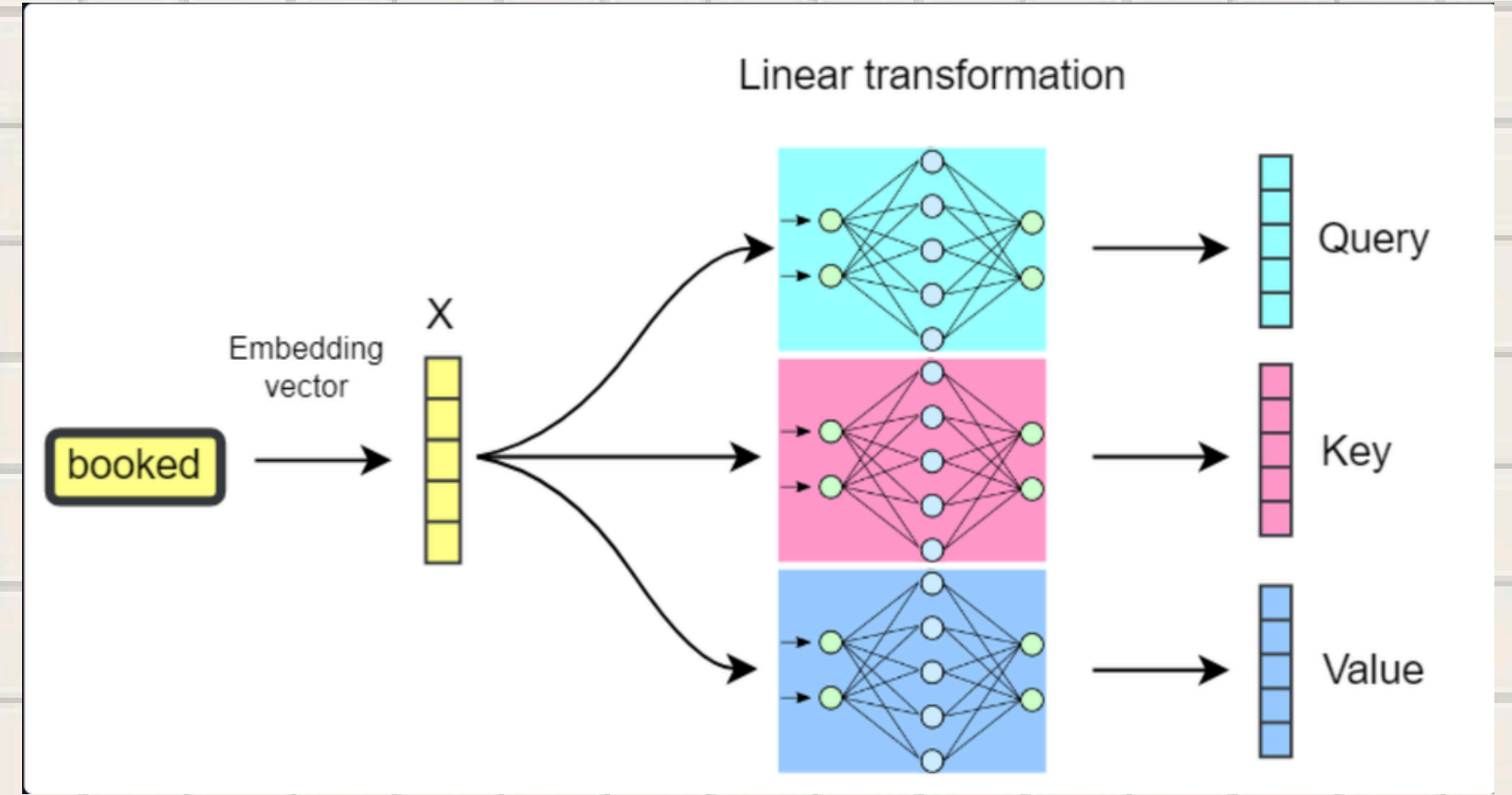
Yararlı ve ilgili başka bir analogi de dictionary (veya hashmap) veri yapısıdır. Bir dictionary, verileri key-value çiftleri halinde saklar ve key'leri kendi ilgili value çiftleriyle eşleştirir. Dictionary'den belirli bir value elde etmeye çalıştığınızda, ilgili key ile eşleşecek bir query sağlamanız gerekir; ardından sistem bu key'ler arasında arama yapar, bunları query ile karşılaştırır ve eşleşme sağlanırsa istenen value döndürülür.

Ancak buradaki fark, bunun bir "kesin eşleşme" (hard-matching) durumu olmasıdır; yani Query ya Key ile tam olarak eşleşir ya da eşleşmez ve aralarında kısmi bir benzerlik derecesi ölçülmez.



Daha önce sadece gerçel değerli vektörlerle (token embedding'leri) çalıştığımızdan bahsetmiştik. Dolayısıyla Query, Key ve Value bileşenlerinin de vektör olması gerekir. Ancak şu ana kadar her bir token için elimizde sadece bir tane vektör var, o da onun embedding vektörüdür. Peki, Query, Key ve Value vektörlerini nasıl elde etmeliyiz?

Bunları, her bir token'ın embedding vektörünün linear projection'larını (yani W_q , W_k , W_v gibi ayrı ağırlık setlerine sahip tek bir dense layer işlemi olan linear transformation'ları) kullanarak oluştururuz. Bu da ilgili Query, Key ve Value vektörlerini elde etmek amacıyla word embedding üzerinde linear transformation yapabilmek için, Query, Key ve Value'nun her biri için öğrenilebilir (learnable) ağırlık vektörleri kullandığımız anlamına gelir.



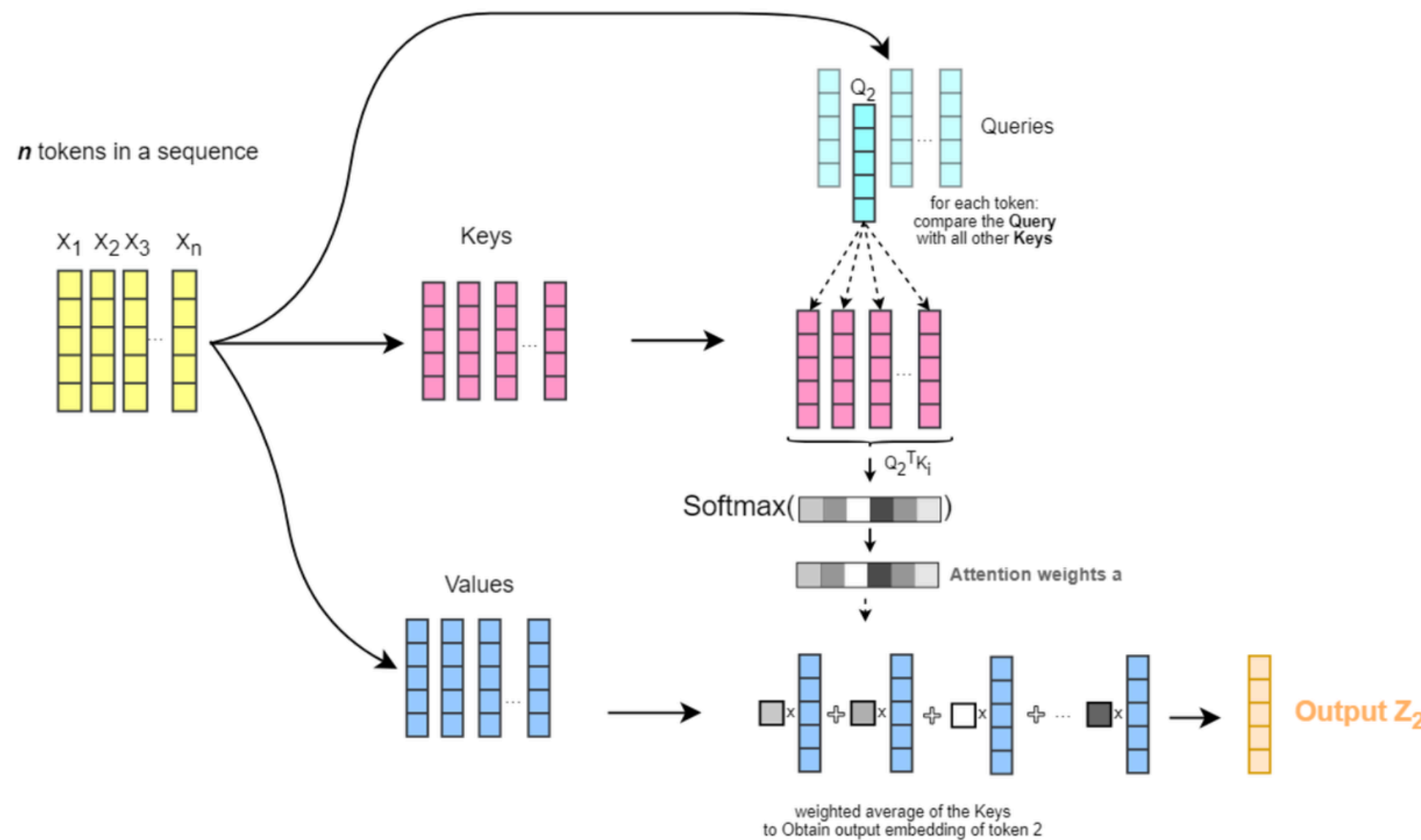
Bir token'ın embedding'i, o token'ın farklı bağlamsal, yapısal ve sözdizimsel özelliklerini temsil edebilir. Bu vektörleri token'ın embedding'inden oluşturmak için öğrenilebilir (learnable) linear transformation katmanları kullanarak, ağırlık vektörlerini yapmasına olanak tanırız:

- Bu bilginin sınırlı ve belirli bir kısmını çıkarıp Q, K ve V vektörlerine aktarmak.
- Arama ve eşleştirme işleminin yapılacağı daha dar bir bağlam belirlemek.
- Bir embedding içindeki hangi bilgiye odaklanmanın (attend) daha önemli olduğunu öğrenmek.

Şimdi, elimizde Q, K ve V vektörleri olduğuna göre, daha önce bahsettiğimiz "arama ve karşılaştırma" işlemi bu vektörlerle gerçekleştirebiliriz. Bu da (Vaswani ve ark., 2017) tarafından önerilen Attention mekanizmasının nihai halini ortaya çıkarır.

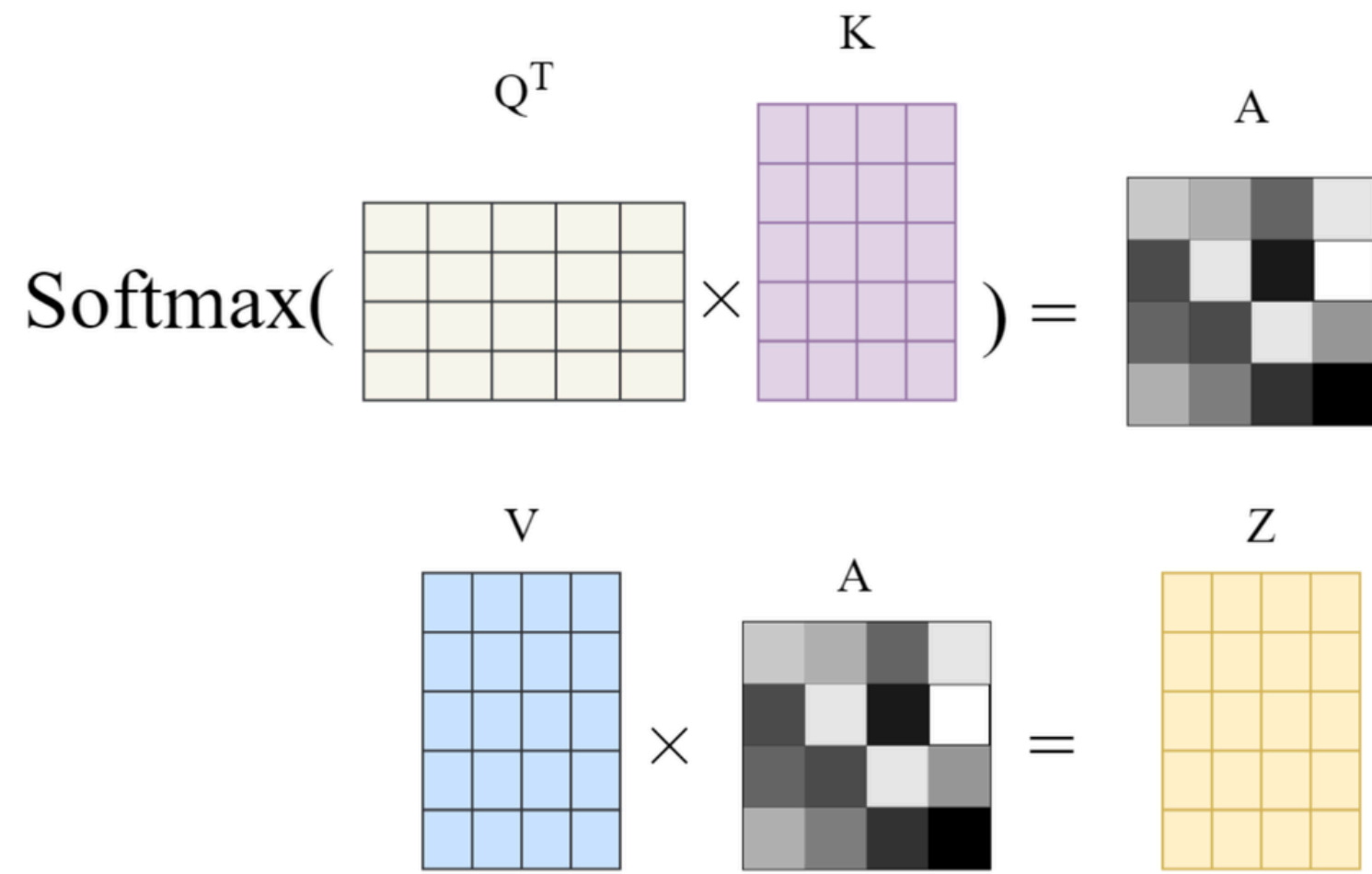
Her bir token için:

- Kendi Query vektörünü diğer tüm token'ların Key vektörleriyle karşılaştırırız.
- Her ikisi arasında bir vektör benzerlik skoru hesaplarız (yani orijinal makaledeki dot-product similarity).
- Bu benzerlik skorlarını $[0,1]$ aralığına ölçeklendirerek ağırlıklara dönüştürürüz (yani Softmax).
- Ve ilgili Value vektörlerini ağırlıklandırarak ağırlıklı bağlamı ekleriz.



Dolayısıyla, Q, K ve V vektörlerinin tüm mantığı, bir dizideki iki token'ın birbiriyle ne kadar ilişkili olduğunu (ağırlıklar) ve bağlam olarak nelerin eklenmesi gerektiğini (value'lar) öğrendiğimiz bir arama ve eşleştirme (search-and-match) sürecini taklit eden esnek (soft) bir dictionary gibidir. Ayrıca, bu sürecin sıralı olarak (her seferinde bir token olacak şekilde) gerçekleşmek zorunda olmadığını unutmayın. Tüm bunlar matris işlemleri kullanılarak paralel bir şekilde gerçekleşir.

Aşağıdaki görselde, matris boyutlarının orijinal makaledekine kıyasla yer değiştirdiğine (dim x n_tokens yerine n_tokens x dim olduğuna) dikkat edin. İlerleyen kısımlarda, Attention mekanizmasının tam tersi yönde olan orijinal ve eksiksiz formülasyonunu göreceksiniz.



Bu durum, eklenen bağlamın token'ların birbirleriyle olan ilgisine dayandığı ve Q , K , V vektör dönüşümleri (transformation) yoluyla öğrenildiği, her bir token için daha bağlamsal (context-aware) bir embedding elde edilmesiyle sonuçlanır. Dolayısıyla buna dot-product attention mekanizması denir.

(Vaswani ve ark., 2017)'deki orijinal Attention mekanizması ayrıca K ve Q vektörlerinin dot-product'ını ölçeklendirir (scale eder); yani elde edilen vektörü, Query vektörünün boyutu olan $\sqrt{d}$ değerine böler. Bu nedenle "scaled dot-product attention" adını alır.

Bu ölçeklendirme işlemi, değerler Softmax fonksiyonuna aktarılmadan önce dot-product'ın varyansını azaltmaya yardımcı olur:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Multi-Head Attention Nedir?

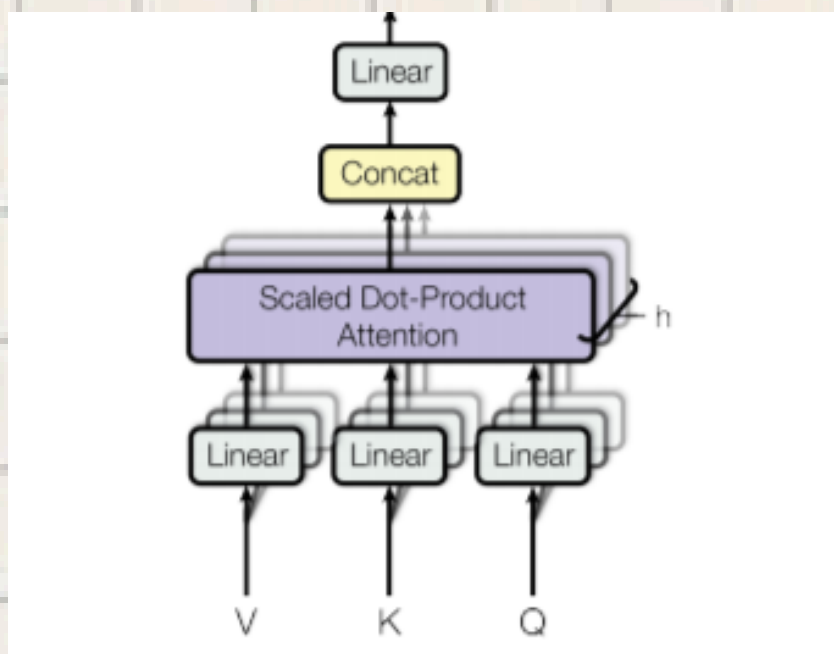
Multi-head attention, girdiyi birden fazla head'e bölerek self-attention mekanizmasını genişletir ve modelin çeşitli ilişkileri ve örüntüleri (pattern) yakalamasını sağlar.

Tek bir Q, K, V matris seti kullanmak yerine, girdi embedding'leri her birinin kendi Q, K ve V bileşenlerine sahip olduğu birden fazla sete (head'e) yansıtılır (project edilir):

- Linear Transformation: X girdisi, farklı ağırlık matrisleri kullanılarak daha küçük boyutlu birden fazla alt uzaya (subspace) yansıtılır.

$$Q_i = XW_i^Q, K_i = XW_i^K, V_i = XW_i^V \quad (\text{burada } i, \text{ head indeksini belirtir.})$$

- Independent Attention Computation: Her bir head, scaled dot-product formülünü kullanarak kendi self-attention değerini bağımsız olarak hesaplar.
- Concatenation: Tüm head'lerden elde edilen çıktılar uç uca eklenerek birleştirilir (concatenate edilir).
- Final Linear Transformation: Birleştirilen bu çıktıyı istenen boyuta dönüştürmek için son bir ağırlık matrisi uygulanır.



Matematiksel olarak, multi-head attention şu şekilde ifade edilir:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h)W^O$$

burada:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Tek head neden yetmez?

Tek başlık kullanıldığında her pozisyon için yalnızca tek bir dikkat ağırlığı dağılımı üretilir. Bu durumda model, bir kelimenin hangi ilişkiyi öncelikle dikkate alması gerektiğine karar vermek zorundadır: Özne-fiil uyumu mu, konum bilgisi mi yoksa zamir eşleştirmesi mi? Örneğin "The lawyer who the witness saw left." cümlesinde who hem lawyer sözcüğüne geri referans verir, hem de saw fiilinin nesnesidir; saw ise witness ile özne-fiil ilişkisi kurar. Tek başlık bu çoklu bağıntıları tek bir vektörde kodlamaya çalıştığı için ilişki türleri arasında seçim yapmak zorundadır. Bu durum, farklı ilişkileri aynı anda yakalamanın zor olduğu anlamına gelir

Multi-head attention çeşitli avantajlar sağlar:

- Farklı ilişkileri yakalar: Farklı head'ler, girdinin farklı yönlerine odaklanır (attend eder).
- Öğrenme verimliliğini artırır: Paralel olarak çalışması sayesinde, birden fazla head bağımlılıkların (dependencies) daha iyi öğrenilmesine olanak tanır.
- Dayanıklılığı (robustness) artırır: Model tek bir attention pattern'ına bağlı kalmaz, bu da overfitting'i (aşırı öğrenmeyi) azaltır.

Multi-Head Attention Nasıl Çalışır?

Multi-head attention, modelin kapasitesini h adet paralel attention head'e böler. Her bir head daha düşük bir boyutta (reduced dimension) işlem yapar, böylece toplam hesaplama yükü, tam boyutlu bir single-head attention ile benzer seviyede kalır.

Süreç dört adımda gerçekleşir:

- Girdiyi h adet ayrı query, key ve value temsillerine yansıtmak (project etmek).
- Her bir head için bağımsız olarak scaled dot-product attention hesaplamak.
- Tüm head çıktılarını tek bir tensor halinde birleştirmek (concatenate etmek).
- Birleştirilmiş (concatenated) çıktıyı yeniden model boyutuna (model dimension) yansıtmak (project etmek).

Şimdi bu adımların her birini somut matematiksel işlemlerle inceleyelim.

1. Adım: Paralel Projections

n adet token ve model dimension d_{model} 'e sahip bir $X \in \mathbb{R}^{n \times d_{model}}$ girdi dizisi verildiğinde, bunu her bir head için **query**, **key** ve **value** değerlerine yansıtırız (project ederiz). Eğer h adet head'imiz varsa, genellikle head dimension'ı $d_k = d_{model} / h$ olarak ayarlarız. Bu bölme işlemi, tüm head'ler üzerindeki toplam hesaplamanın, tam boyut üzerindeki tek bir **attention** işlemiyle kıyaslanabilir seviyede kalmasını sağlar.

i indeksli head için, öğrenilmiş (learned) linear projection'lar aracılığıyla **query**, **key** ve **value** matrislerini şu şekilde hesaplarız:

$X \in \mathbb{R}^{n \times d_{model}}$: her biri d_{model} boyutlu bir vektör olarak temsil edilen n adet token'a sahip girdi dizisi.

$W_i^Q \in \mathbb{R}^{d_{model} \times d_k}$: i head'i için her bir token'ı d_k boyutlu bir **query** vektörüne dönüştüren öğrenilmiş **query projection matrix**'i.

$W_i^K \in \mathbb{R}^{d_{model} \times d_k}$: i head'i için her bir token'ı d_k boyutlu bir **key** vektörüne dönüştüren öğrenilmiş **key projection matrix**'i.

$W_i^V \in \mathbb{R}^{d_{model} \times d_v}$: i head'i için öğrenilmiş **value projection matrix**'i (genellikle $d_v = d_k$).

$Q_i, K_i \in \mathbb{R}^{n \times d_k}$: i head'i için elde edilen **query** ve **key** matrisleri.

$V_i \in \mathbb{R}^{n \times d_v}$: i head'i için elde edilen **value** matrisi.

d_k : her bir head'in boyutu, genellikle d_{model} / h .

Her head kendi projection matrislerine sahip olur ve bu da girdinin farklı dönüşümlerini (transformations) öğrenmesine olanak tanır. 1. head sözdizimsel (syntactic) özellikleri vurgulamak için token'ları yansıtırken, 2. head anlamsal (semantic) benzerliği vurgulayabilir. Buradaki asıl (key) fikir, farklı projection matrislerinin aynı girdinin farklı "görünümlerini" (views) oluşturması ve her birinin potansiyel olarak token ilişkilerinin farklı yönlerini öne çıkarmasıdır.

2. Adım: Independent Attention Computation

Her bir head, kendi projection'larını kullanarak standart scaled dot-product attention hesaplar. Attention mechanism, ilgi ağırlıklarını (relevance weights) belirlemek için query'leri key'lerle karşılaştırır, ardından bu ağırlıkları value'ları bir araya getirmek (aggregate) için kullanır:

$$head_i = Attention(Q_i, K_i, V_i) = softmax \left(\frac{Q_i K_i^T}{\sqrt{d_k}} \right) V_i$$

burada:

$head_i \in \mathbb{R}^{n \times d_v}$ tüm n adet token için bağlamsal temsilleri (contextual representations) içeren i attention head'inin çıktısı.

$Q_i K_i^T \in \mathbb{R}^{n \times n}$ (j,k) girdisinin, j konumundaki query'nin k konumundaki key ile ne kadar eşleştiğini ölçtüğü ham attention skorları matrisi.

$\sqrt{d_k}: d_k$ boyutu arttıkça dot product'ların çok fazla büyümesini önleyen ölçeklendirme faktörü (scaling factor) (önceki scaled dot-product attention bölümünde ele alınmıştı).

$softmax(\cdot)$ skorları, her bir query konumu için toplamı 1 olacak şekilde attention weights'e dönüştürmek amacıyla satır bazında (row-wise) uygulanır.

V_i satırları attention weights'e göre bir araya getirilen (aggregated) value matrisi.

Bu işlem, her biri $\mathbb{R}^{n \times d_v}$ boyutunda olan h adet ayrı çıktı üretir. Head'ler bu aşamada tamamen bağımsız çalışır; aralarında hiçbir bilgi paylaşımı yoktur. Bu bağımsızlık, farklı head'lerin farklı ilişki türlerini tespit etmede uzmanlaşmasına olanak tanıyan asıl özelliktir.

3. Adım: Concatenation

Tüm head'leri hesapladıktan sonra, çıktılarını feature dimension boyunca uç uca ekleriz (concatenate ederiz). Her bir head bir $n \times d_v$ matrisi üretir ve bunları yan yana eklemek daha büyük bir matris oluşturur:

$$\text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h) \in \mathbb{R}^{n \times (h \cdot d_v)}$$

burada:

$\text{head}_1, \dots, \text{head}_h$: her biri $\mathbb{R}^{n \times d_v}$ boyutunda olan h adet bireysel head çıktısı.

$h \cdot d_v$: tüm head'lerin çıktılarını bir araya getiren concatenated feature dimension.

Eğer $d_v = d_k = d_{\text{model}}/h$ ise, o zaman $h \cdot d_v = d_{\text{model}}$ olur ve birleştirilmiş (concatenated) çıktı, orijinal girdi ile aynı boyuta sahip olur. Bu boyutsal tutarlılık kasıtlıdır: multi-head attention'ın, single-head attention yerine doğrudan bir alternatif (drop-in replacement) olarak kullanılabilmesine olanak tanır. Concatenation işlemi her bir head'in katkısını ayrı ayrı korur ve bir sonraki projection'ın bunları nasıl birleştireceğini öğrenmesini sağlar.

4. Adım: Output Projection

Son olarak, concatenate edilmiş (birleştirilmiş) çıktıya öğrenilmiş (learned) bir linear projection uyguluyoruz. Bu projection, birleştirilen head çıktılarını yeniden model dimension'ına eşler (map eder):

$$MultiHead(X) = Concat(head_1, \dots, head_h)W^O$$

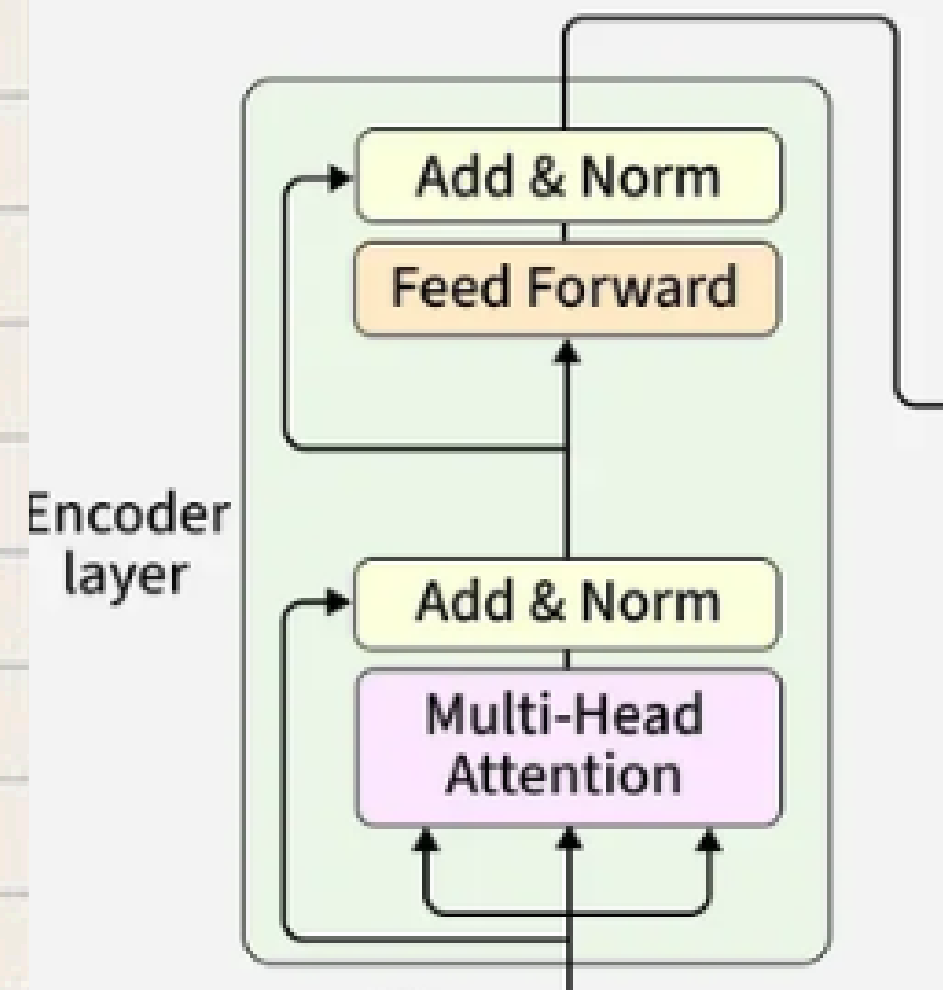
burada:

$Concat(head_1, \dots, head_h) \in \mathbb{R}^{n \times (h \cdot d_v)}$: tüm h adet **attention head**'inden elde edilen *concatenated* çıktılar.

$W^O \in \mathbb{R}^{(h \cdot d_v) \times d_{model}}$: öğrenilmiş **output projection** matrisi.

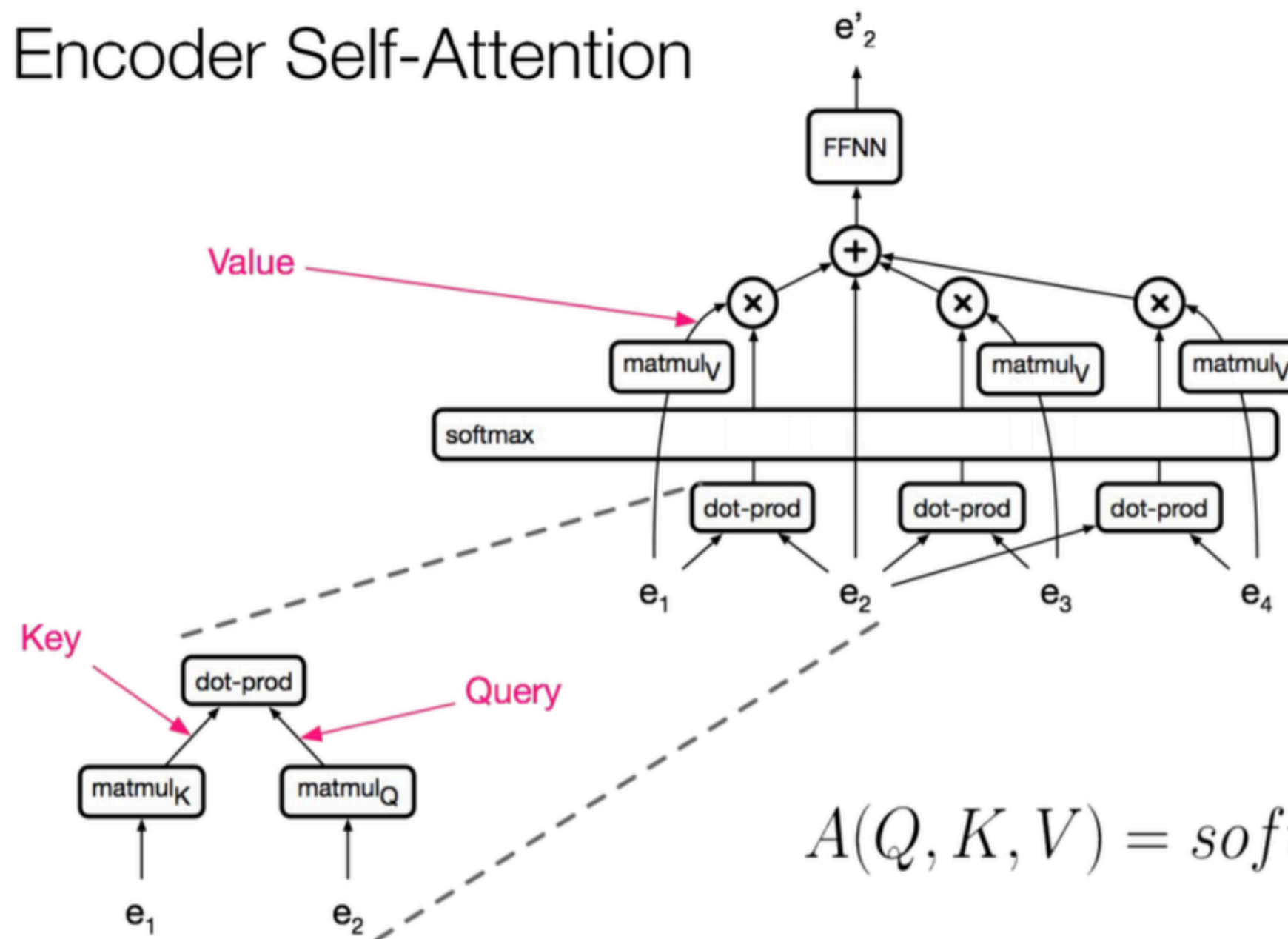
$MultiHead(X) \in \mathbb{R}^{n \times d_{model}}$: girdi boyutuyla eşleşen nihai çıktı.

Bu projection iki amaca hizmet eder. İlk olarak, tüm head'lerden gelen bilgileri birleştirilerek modelin bu katkıları nasıl harmanlayacağını (mix edeceğini) öğrenmesine olanak tanır. Syntax (sözdizimi) konusunda uzmanlaşmış bir head belirli token'lar için daha ağır basarken, diğerlerinde semantic (anlamsal) bir head baskın olabilir. İkinci olarak, çıktı boyutunun girdi boyutuyla tam olarak eşleşmesini sağlayarak, transformer mimarisindeki residual connections kullanımını mümkün kılar.



Encoder Self-Attention: Bu, encoder'ın girdi dizisi (input sequence) içindeki bağlamsal ilişkileri öğrenmesine olanak tanır. Girdideki her bir kelime (veya token), diğer tüm kelimelere odaklanır (attend eder); bu da modelin, dizideki mesafelerinden bağımsız olarak kelimeler arasındaki bağımlılıkları (dependencies) anlamasına yardımcı olur.

Encoder Self-Attention

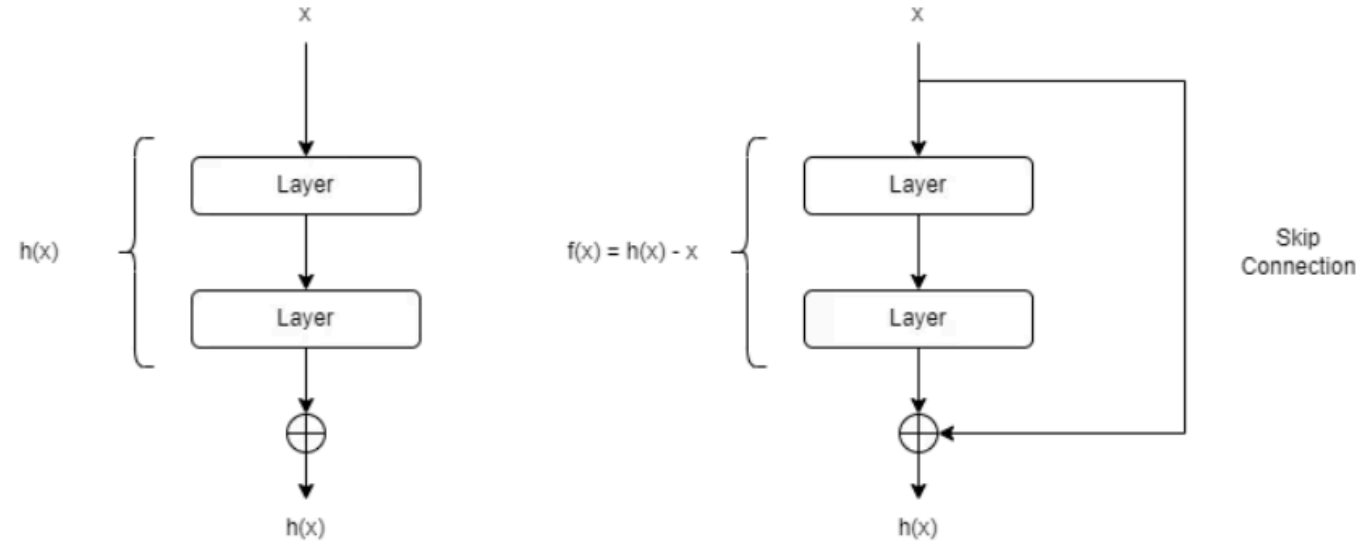


$$A(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Residual Connection (Add)

Transformer ağlarında Residual Connection (Add)

avantajı, hem ileri hem de geri yollarda sinyal yayılımını kolaylaştırarak kaybolan gradyanlar sorununu çözmesidir. Transformer gibi daha derin katmanlara sahip karmaşık ağlar, kaybolan gradyan sorunlarından muzdariptir ve atlama bağlantısı, özdeşlik eşlemesi yaparak azalan gradyanları geri kazanmaya yardımcı olur.



Şekil 2 ve 3: Geleneksel Öğrenme ve Artık Öğrenme [2]

Residual Connection (Add)

, bu alt ağı yeniden parametrelendirir ve parametre katmanlarının bir "artık fonksiyonu" temsil etmesine olanak tanır. Başka bir deyişle, model, altta yatan fonksiyon $h(x)$ ' i yaklaşık olarak hesaplamak yerine , bir artık fonksiyon $f(x)$ ' i yaklaşık olarak hesaplamaya zorlanacaktır .

İdeal olarak, $y = h(x)$. Artık bağlantıdan sonra, $y = h(x) + x$

Dolayısıyla, $h(x)$ ' i tahmin etmek yerine , $f(x) = h(x) - x$ 'i tahmin ediyoruz

.Şimdilik FP için genel ilişkimiz şu şekilde olacak:

$$y = f(x) + x$$

Geriye doğru yayılım

Bu noktada artık bağlantı, kaybolan gradyan sorununu ele almaktadır. Gradyanları hesaplamak için, bir kayıp fonksiyonu ε 'nin giriş x 'e göre kısmi türevini ele alacağız . Yukarıdaki formülü kullanarak geri yayılım denklemimiz şu şekilde olacaktır: Bu denklemle, $\partial f(x)$ gradyanı azalsa bile, fazladan bir $\partial \varepsilon / \partial y$ nedeniyle toplam gradyan sifıra inmez.

$$\frac{\partial \varepsilon}{\partial x} = \frac{\partial \varepsilon}{\partial y} \cdot \frac{\partial y}{\partial x} = \frac{\partial \varepsilon}{\partial y} \cdot \left(\frac{\partial f(x)}{\partial x} + 1 \right)$$

Denklem 1: Artık öğrenmede geri yayılım

Layer Normalization (Norm)

Layer Normalization, önceki işlemin çıktısını özellikler genelinde normalleştirir . Layer Normalization sırasında, toplu işlemdeki her öge 0-1 aralığında normalize edilir. Bu öğeler herhangi bir şey olabilir; transformatörler söz konusu olduğunda, bir kelimenin 512 boyutlu vektör gömülmesi olabilir. Bu öğelerin ortalamasını ve varyansını hesaplıyoruz ve matristeki her değeri bununla değiştiriyoruz.

$$\hat{x}_j = \frac{x_j - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

Denklem 2: Katman Normalizasyonu

Transformer'larda Katman Normalizasyonunun Eksiklikleri

Katman normalizasyonu çeşitli avantajlar sağlarken, bazı sınırlamaları da vardır. Başlıca dezavantajlarından biri, verilerin 0-1 ölçeğinde normalize edilmesiyle modelin çok kısıtlayıcı hale gelmesidir. Bu sınırlama, modelin etkili bir şekilde öğrenme yeteneğini engelleyebilir.

Bu sorunu hafifletmek için, orijinal Transformer makalesi iki öğrenilebilir parametre sunmaktadır:

- Gama (Çarpımsal)
- Beta (Toplayıcı)

Bu parametreler öğrenilebilir niteliktedir, yani model bunları eğitim sırasında ayarlar. Gama ve Beta, modelin belirli değerleri ne zaman artıracakını veya azaltacağını belirlemesine olanak tanıyarak verilerde gerekli dalgalanmaları oluşturur.

Bu parametreler neden önemli?

- Gama (γ) : Bu parametre, normalleştirilmiş çıktıyı ölçeklendirir. Uygun ölçeklendirme faktörünü öğrenerek, model normalleştirilmiş değerleri, söz konusu görev için daha uygun bir aralığa ayarlayabilir.
- Beta (β) : Bu parametre normalleştirilmiş çıktıyı kaydırır. Modelin normalleştirilmiş çıktıya belirli bir değer eklemesine olanak tanıyarak verilerin ortalamasını ayarlama esnekliği sağlar.

Bu iki parametre eklendikten sonra normalizasyon denklemi şu şekilde olacaktır:

$$\hat{x}_j = \frac{x_j - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}} \cdot \gamma + \beta$$

Denklem 3: Öğrenilebilir parametrelerle Katman Normalizasyonu

Bu parametreleri öğrenerek, model tüm değerlerin 0-1 arasında olması kısıtlamasından kaçınır. Bunun yerine, gerektiğinde dalgalanmalar ekleyerek daha dinamik ve uyarlanabilir bir öğrenme sürecine olanak tanır. Bu, modelin verilerdeki karmaşık kalıpları yakalama ve temsil etme yeteneğini artırır.

Sonuç olarak, artık bağlantıları ve normalizasyonu birleştiren Add & Norm katmanı, Transformer modellerinde çok önemli bir rol oynar. Artık bağlantı (Add), gradyan akışını iyileştirir ve gradyanların ağıdan daha etkili bir şekilde geçmesine izin vererek öğrenmeyi optimize eder. Bu arada, normalizasyon gradyanları stabilize etmeye, genelleme yeteneğini geliştirmeye ve eğitim sürecini hızlandırmaya yardımcı olur. Bu bileşenler birlikte, modelin daha derin mimarileri daha verimli ve etkili bir şekilde eğitmesini sağlayarak, kaybolan gradyanlar ve kısıtlayıcı veri ölçekleri gibi sorunları ele alır.

TRANSFORMERLARDA FFNN

Transformatör mimarisinde Multi-Head Attention mekanizması kelimeler arasındaki bağlamsal ilişkileri kurmakla görevliyken; hemen ardından gelen Konum-Bazlı İleri Beslemeli Ağ (Position-wise FFNN), bu ilişkilerin her bir kelime özelinde derinlemesine işlenmesini sağlar. Attention katmanından gelen doğrusal bilgi harmanı, bu katmanın içindeki Çok Katmanlı Algılayıcı (MLP) mimarisi ve ReLU/GELU gibi aktivasyon fonksiyonları sayesinde doğrusal olmayan (non-linear) bir dönüşüme uğrar. Vektör boyutunun önce genişletilip (örneğin 512'den 2048'e) ardından tekrar orijinal boyutuna daraltılması işlemi, modelin dildeki karmaşık ve soyut özellikleri filtreleyerek öğrenmesine olanak tanır. Kısacası Attention katmanı bilginin 'nereden' geleceğini belirlerken, Feed Forward katmanı o bilginin kelime için 'ne anlama' geldiğini sabitler."

İş Bölümü (Bağlam vs. İçselleştirme): Multi-Head Attention mekanizması kelimeler arası bağlamsal ilişkileri kurarken; Konum-Bazlı FFNN (Position-wise FFNN) her kelimenin bu ilişkileri diğerlerinden bağımsız, kendi özelinde derinlemesine işlediği aşamadır.

Doğrusal Olmayan (Non-Linear) Karmaşıklık: Attention katmanından gelen doğrusal (linear) bilgi harmanı, FFNN içindeki Çok Katmanlı Algılayıcı (MLP) mimarisi ve ReLU/GELU aktivasyon fonksiyonları sayesinde doğrusal olmayan bir zekaya dönüşür. Bu sayede model, dildeki soyut ve karmaşık yapıları öğrenebilir.

Özellik Filtreleme (Genişletme ve Daraltma): Vektör boyutu önce genişletilerek (örn. 512'den 2048'e) kelimenin tüm potansiyel özellikleri açığa çıkarılır. Ardından aktivasyon filtresinden geçirilip tekrar orijinal boyutuna (512'ye) daraltılarak gereksiz bilgiler elenir.

Decoder Bloğu

1. Maskeli Çoklu Başlıklı Dikkat (gelecekteki tokenleri görmeme)

- Decoder katmanındaki self-attention, otomatik üretimin koşullu doğasını korumak için gelecekteki pozisyonlara bakmaz. Yani bir pozisyonun tahmini yalnızca kendisinden önceki pozisyonlara dayanır. Aksi takdirde model, gelecek bilgiyi "cheating" yaparak doğrudan kopyalayabilir.
- Scaled dot-product attention formülü:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Maskeli versiyonunda, bu skorlara bir mask matrisi M eklenir:

$$\text{MaskedAttention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right) V$$

- Mask matrisi genellikle üst üçgen (causal) biçimindedir: pozisyon i için sütun $j > i$ olan girdiler $-\infty$ (ör. -inf) ile maskeleyip geri kalan sıfır yapılır. Bu sayede softmax sonrası maskeleme yapılan pozisyonların ağırlıkları yaklaşık 0 olur. Örneğin 4 token'lık bir sırada mask şu şekildedir:

```
[[0, -inf, -inf, -inf],  
 [0,  0, -inf, -inf],  
 [0,  0,  0, -inf],  
 [0,  0,  0,  0  ]]
```

Buradaki 0'lar izin verilen dikkat bağlantılarını, $-\infty$ ise engellenen (gelecek) bağlantıları gösterir. Sonuçta, future pozisyonlar softmax'ten sonra etkisizleşir

2. apraz Dikkat - Encoder ıktıları ile Etkileşim

- Decoder katmanında "apraz dikkat" (cross-attention) katmanı bulunur. Burada decoder'ın nceki alt katmanından gelen Query (sorgu) vektrleri ile encoder'ın en st katmanından gelen Key/Value vektrleri bir araya gelir



- Matematiksel olarak:

$$\text{Attention}(Q_{dec}, K_{enc}, V_{enc}) = \text{softmax}\left(\frac{Q_{dec}K_{enc}^T}{\sqrt{d_k}}\right) V_{enc}$$

formlyle hesaplanır. Bu, decoder'ın sorgusunun encoder ıktıları zerinde nasıl odaklanacaėını belirler.

- apraz dikkat sırasında maske kullanılmaz: Encoder'dan gelen tm anahtar/deėerlar (input dizisi) zaten bilindiėi iin decoder, encoder'ın tm pozisyonlarına aynı anda bakabilir. Bu nedenle herhangi bir bilgi akışı kısıtlaması yoktur; decoder, encoder ıktısındaki ilgilendiėi blgelere zgrce dikkat edebilir. Şekilde eviri rneėinde de (stte) decoder, "how are you?" girdisinin tm paralarını gz nnde bulundurarak ıktıyı retmektedir.

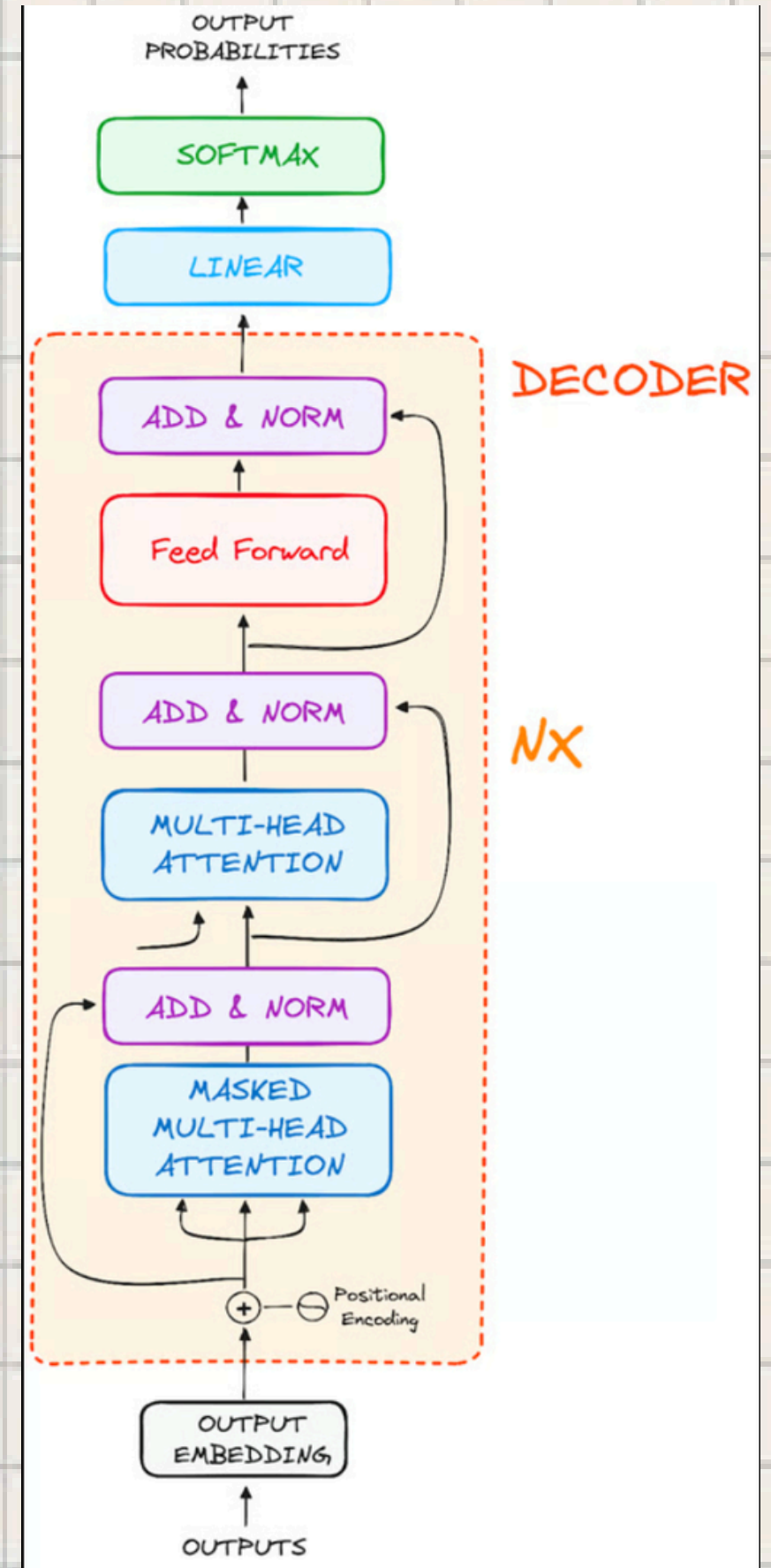
3. Feed-Forward + Toplama & Normalizasyon

- Her decoder katmanında dikkat katmanlarından sonra bir Feed-Forward Ağı uygulanır. Bu katman, her token için bağımsız şekilde iki lineer dönüşüm ve ortasında ReLU aktivasyonu içerir.

Formülü şöyledir: $FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$

Aynı W_1 , b_1 , W_2 parametreleri tüm pozisyonlar için tekrar tekrar kullanılır (pozisyon başına aynı dönüşüm).

- Ardından residual bağlantı ve katman normalizasyonu (LayerNorm) uygulanır. Yani katmana gelen giriş x ile katman çıktısı toplanır ve sonuç normalize edilir: $y = \text{LayerNorm}(x + \text{Sublayer}(x))$ Bu mekanizma, derin ağlarda gradyan akışını kolaylaştırır ve eğitim stabilitesini artırır. Her alt katmanın çıkışında aynı şekilde topla-normalize yapılır



4. Oto-regresif Üretim Mantığı

- Teacher Forcing (Eğitim aşaması): Eğitim sırasında model, bir sonraki token'ı tahmin ederken önceki doğru çıktıları giriş olarak alır. Yani girdi olarak gerçek çıktı dizisi (sağa kaydırılmış) verilir. Bu sayede her adımda model yalnızca geçmiş bilgilere dayanarak öğrenir ve gelecek bilgiyi doğrudan kopyalamaktan kaçınır.
- Üretim (Inference) aşaması: Üretilen çıktılar bilinmediğinden model birer birer token üretir. Her adımda önceki tahmin (örn. y_{i-1}) decoğder'ın girdisine eklenir. Bu süreç teacher forcing'e göre farklıdır: gerçek çıkış yerine modelin kendi önceki tahminleri kullanılır
- Yeniden Hesaplama ve Karmaşıklık: Üretim sırasında decoder her token ürettiğinde öncekilerle birlikte self-attention hesaplaması tekrar yapılır. Bu hesaplamalar n token uzunluğunda bir dizide $n \times n$ işlemeyi gerektirdiğinden zaman ve bellek karmaşıklığı $O(n^2)$ olur. Özetle, örneğin 100 tokenlık bir dizide dikkatin her pozisyonundan her pozisyona bakması gerektiği için hesaplama karesel olarak artar. Bu nedenle uzun dizilerde işlem maliyeti hızla yükselir